

방향성 멀티그래프를 위한 증명 가능한 강력한 그래프 신경망

Béni Egressy^{1*}, Luc von Niederhäusern¹, Jovan Blanuša², Erik Altman³, Roger Wattenhofer¹,
Kubilay Atasu²

¹ETH Zurich, 스위스 취리히 | ²IBM Research Europe, 스위스 취리히 | ³IBM Watson Research, 미국 뉴욕
Yorktown Heights

* 이 연구는 Béni Egressy와 Luc von Niederhäusern이 IBM Research Europe(취리히)에 재직 중일 때 수행되었습니다. AAAI 2024 학회 논문으로 게재 예정. (arXiv:2306.11586v3 [cs.LG], 2024년 1월 4일)

초록

이 논문은 표준 메시지 전달 그래프 신경망(GNN)을 증명 가능한 강력한 방향성 멀티그래프 신경망으로 변환하는 간단한 적응(adaptation) 기법들의 집합을 분석한다. 이 적응 기법에는 멀티그래프 포트 번호(port numbering), ego ID, 그리고 역방향 메시지 전달(reverse message passing)이 포함된다. 우리는 이 기법들의 조합이 이론적으로는 어떤 방향성 부분그래프 패턴이든 탐지할 수 있게 해준다는 것을 증명한다. 제안한 적응 기법들의 실제 효과를 검증하기 위해 합성 부분그래프 탐지 과제에 대한 실험을 수행하였으며, 거의 완벽에 가까운 뛰어난 성능을 보였다.

또한 제안한 적응 기법들을 두 가지 금융 범죄 분석 과제에 적용하였다. 자금세탁 거래 탐지에서 표준 메시지 전달 GNN의 소수 클래스 F1 점수를 최대 30% 향상시키는 극적인 개선을 관찰했으며, 트리 기반 및 GNN 베이스라인과 거의 대등하거나 이를 능가하는 성능을 보였다. 실제 피싱 탐지 데이터셋에서도 비슷하게 인상적인 결과를 관찰했는데, 세 가지 표준 GNN의 F1 점수를 약 15% 향상시키고 모든 베이스라인을 능가하였다.

1. 서론

그래프 신경망(GNN)은 관계형 데이터로부터 학습하는 데 있어 사실상 표준이 되는 머신러닝 모델이 되었다. GNN은 생물학, 물리학, 화학에서부터 소셜 네트워크, 교통, 일기 예보에 이르기까지 다양한 분야에서 사용되고 있다(Bongini, Bianchini, and Scarselli 2021; Zhou et al. 2020; Derrow-Pinion et al. 2021; Shu, Wang, and Liu 2019; Wu et al. 2020; Keisler 2022; Zhang et al. 2019; Battaglia et al. 2016). 더 최근에는 GNN을 금융 범죄 식별에 활용하려는 관심이 커지고 있다(Cardoso, Saleiro, and Bizarro 2022; Kanezashi et al. 2022; Weber et al. 2019, 2018; Nicholls, Kuppa, and Le-Khac 2021).

우리의 동기가 된 과제는 거래 네트워크에서 부분그래프 패턴으로 나타나는 금융 범죄를 탐지하는 것이다. 예를 들어, 확립된 자금세탁 패턴을 묘사한 그림 1이나, 부록의 예시 시나리오를 보여주는 그림 4를 참조하라. 그러나 유사한 패턴은 화학에서부터 교통 예측에 이르기까지 많은 분야의 그래프 과제와도 관련이 있다는 점에 주목하라. 이 과제는 얼핏 보기에 GNN을 사용하기에 매우 적합해 보인다. 그러나 안타깝게도 현재의 GNN은 금융 거래 네트워크를 효과적으로 다루기에 부족한 점이 있다.

첫째, 금융 거래 네트워크는 사실 방향성 멀티그래프이다. 즉, 엣지(거래)는 방향을 가지며, 두 노드(계좌) 사이에 여러 개의 엣지가 존재할 수 있다. 둘째, 대부분의 GNN은 사이클과 같은 일부 부분그래프 패턴을 탐지할 수 없다(Chen et al. 2020, 2019b). 이 한계를 극복하기 위한 많은 노력이 있었지만(You et al. 2021; Huang et al. 2022; Papp and Wattenhofer 2022; Zhang and Li 2021; Loukas 2019; Sato, Yamada, and Kashima 2019), 모두 단순 (무방향) 그래프에 초점을 맞추고 있었다. 그러나 단순 그래프에서조차 이 문제는 해결되었다고 보기 어렵다. 예를 들어, 아주 최근까지도 이론적 보장과 함께 6-사이클을 셀 수 있는 선형 시간 순열 동변(permutation-equivariant) GNN이 존재하지 않았다(Huang et al. 2022).

이 논문은 이 두 가지 문제를 모두 다룬다. 우리가 아는 한, 이것은 방향성 멀티그래프를 위해 특별히 설계된 최초의 GNN 아키텍처이다. 둘째로, 우리는 먼저 제안된 아키텍처가 이론적으로 방향성 멀티그래프의 어떤 부분그래프 패턴이든 탐지할 수 있음을 증명한 다음, 제안 아키텍처가 그림 1에 묘사된 패턴들을 탐지할 수 있음을 경험적으로 확인한다. 제안 아키텍처는 어떤 표준 GNN 아키텍처든 방향성 멀티그래프 GNN으로 변환할 수 있는 간단한 적응 기법들의 집합에 기반한다. 이 적응 기법들은 역방향 메시지 전달(Jaume et al. 2019), 포트 번호(Sato, Yamada, and Kashima 2019), 그리고 ego ID(You et al. 2021)이다. 이 개별 구성 요소들은 기존 문헌에 이미 존재하지만, 이들을 결합했을 때의 이론적·경험적 힘은 탐구된 적이 없다. 본 연구에서 우리는 이 공백을 메운다. 즉, 이들을 결합하고 방향성 멀티그래프에 맞게 적응시키며, 함께 사용했을 때의 이론적·경험적 이점을 보여준다.

우리의 기여. (1) 메시지 전달 GNN을 증명 가능한 강력한 방향성 멀티그래프 신경망으로 변환할 수 있는 간단하고 직관적인 적응 기법들의 집합을 제안한다. (2) ego ID, 포트 번호, 역방향 메시지 전달을 갖춘 충분히 강력한 GNN이 어떤 방향성 부분그래프 패턴이든 식별할 수 있음을 증명한다. (3) 이론을 합성 그래프에서 검증하여, 이 적응 기법들을 사용하는 GNN이 길이 6까지의 방향성 사이클, 스캐터-개더(scatter-gather) 패턴, 방향성 바이클릭(biclique)을 포함한 다양한 부분그래프 패턴을 탐지할 수 있음을 확인함으로써 기존 GNN 아키텍처와 차별화된다. (4) 이러한 개선은 두 금융 데이터셋에서 상당한 성능 향상으로 이어진다. 이 적응 기법들은 자금세탁 및 피싱 데이터셋에서 GNN의 성능을 극적으로 향상시켜, 시뮬레이션 데이터와 실제 데이터 모두에서 최첨단 금융 범죄 탐지 모델과 대등하거나 이를 능가한다.

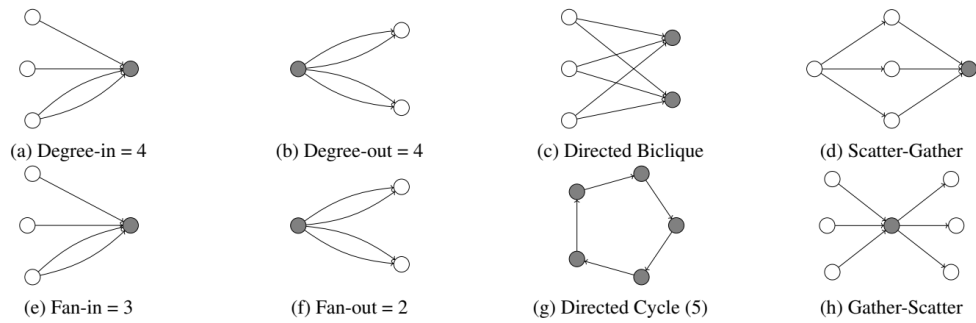


그림 1: 자급세탁 패턴. 회색 채움은 합성 패턴 탐지 과정에서 탐지 대상이 되는 노드를 나타낸다. 여기서 나타난 구체적인 차수/팬 패턴 크기는 예시를 위한 것일 뿐이다.

2. 관련 연구

Xu et al. (2018)은 표준 MPNN이 기껏해야 Weisfeiler-Lehman(WL) 동형(isomorphism) 테스트만큼만 강력하다는 것을 보였고, 이론적으로 WL 테스트의 힘과 일치하는 GNN 아키텍처인 GIN을 제시하였다. WL 테스트는 점근적으로 거의 확실하게 임의의 두 비동형 그래프를 구별할 수 있지만(Babai, Erdos, and Selkow 1980), 표준 MPNN은 특정 그래프에서는 사이클과 같은 단순한 부분구조조차 탐지할 수 없다(Chen et al. 2020, 2019b). 이는 연구자들이 표준 MPNN을 넘어서도록 동기를 부여하였다.

한 방향은 k -튜플 간에 메시지 전달을 수행하거나 텐서 기반 모델을 사용하여 더 강력한 k -WL 동형 테스트를 모방하는 것이다(Maron et al. 2019; Morris et al. 2019). 안타깝게도 이러한 모델은 복잡도가 높아 대부분의 응용에서 비실용적이다. 또 다른 계열의 연구는 사전 계산된 특성을 사용하여 GNN을 보강한다. 이러한 연구들은 부분그래프 개수(Bouritsas et al. 2022; Barceló et al. 2021), 위치적 노드 임베딩(Egressy and Wattenhofer 2022; Dwivedi et al. 2021), 랜덤 ID(Abboud et al. 2020; Sato, Yamada, and Kashima 2021), 노드 ID(Loukas 2019)의 추가를 탐구한다.

부분그래프 GNN(Subgraph GNN)이라 불리는 최근의 표현력 있는 GNN 계열은 그래프를 부분그래프의 모음으로 모델링한다(Frasca et al. 2022; Zhao et al. 2021). Papp et al. (2021)은 입력에서 랜덤 노드를 제거하고 GNN을 여러 번 실행하여 실행할 때마다 더 많은 정보를 수집한다. Zhang and Li (2021)은 대신 각 노드 주변의 부분그래프를 추출하여 그 위에서 GNN을 실행한다. 이 범주에 속하는 것으로 ego ID를 사용하는 ID-GNN도 있는데(You et al. 2021), 각 노드를 이웃과 함께 샘플링하고 이웃과 구별하기 위한 식별자를 부여한다. 저자들은 ID-GNN이 사이클을 셀 수 있다고 주장하지만, 그 증명은 부정확한 것으로 밝혀졌다. 실제로 Huang et al. (2022)은 부분그래프 GNN 계열 전체가 길이 4보다 긴 사이클을 셀 수 없다는 것을 보였고, 길이 6까지의 사이클을 셀 수 있는 I^2 -GNN을 제안하였다.

방향성 그래프를 위한 GNN에 대한 연구는 훨씬 적었다. Zhang et al. (2021)은 방향성 그래프를 위한 스펙트럴 네트워크를 제안했지만, 이 네트워크의 힘을 분석하거나 더 큰 데이터셋에 적용하기는 어렵다. 유사한 접근법은 (Tong et al. 2020)과 (Ma et al. 2019)에서 찾을 수 있다. Jaume et al. (2019)은 그래프를 무방향으로 순진하게 취급하는 대신 들어오는 이웃과 나가는 이웃을 별도로 집계하도록 메시지 전달을 확장하였다. 방향성 멀티그래프는 구체적으로 고려된 적이 없다.

GNN은 다양한 금융 응용에 사용되어 왔다(Li et al. 2021; Feng et al. 2019; Chen, Wei, and Huang 2018; Zhang et al. 2019; Li et al. 2019; Xu et al. 2021; Yang et al. 2021). 우리의 연구와 가장 가까운 것은 사기 탐지에 GNN을 사용한 연구들이다. Liang et al. (2019)과 Rao et al. (2021)는 이분 고객-상품 그래프에서 각각 보험 사기와 신용카드 사기를 밝혀내는 연구를 수행하였다. Liu et al. (2018)은 온라인 결제 플랫폼의 기기-활동 이분 그래프에서 악성 계정을 탐지하기 위해 이종(heterogeneous) GNN을 사용하였다. Weber et al. (2019)은 표준 GNN을 반자금세탁(AML)에 최초로 적용하였고, 더 최근에는 Cardoso, Saleiro, and Bizarro (2022)가 거래 네트워크를 이분 계좌-거래 그래프로 표현할 것을 제안하여 반지도 AML 환경에서 유망한 결과를 보였다. 그러나 이러한 접근법이 전형적인 사기 패턴 탐지에 어떻게 도움이 되는지는 명확하지 않다.

3. 배경

3.1 그래프와 금융 거래 그래프

우리는 방향성 멀티그래프 G 를 고려한다. 여기서 노드 $v \in V(G)$ 는 계좌를 나타내고, 방향성 엣지 $e = (u, v) \in E(G)$ 는 u 에서 v 로의 거래를 나타낸다. 각 노드 u 는 (선택적으로) 계좌 특성의 집합 $h^{(0)}(u)$ 를 가지며, 여기에는 계좌 번호, 은행 ID, 계좌 잔액 등이 포함될 수 있다. 각 거래 $e = (u, v)$ 는 연관된 거래 특성의 집합 $h^{(0)}_{(u,v)}$ 를 가지며, 여기에는 거래 금액, 통화, 타임스탬프가 포함된다. u 의 들어오는 이웃과 나가는 이웃은 각각 $N_{in}(u)$ 과 $N_{out}(u)$ 으로 표기한다. 동일한 두 계좌 사이에는 여러 거래가 가능하며, 이로 인해 G 는 멀티그래프가 된다. 노드(또는 엣지) 예측 과제에서 각 노드(또는 엣지)는 해당 계좌(또는 거래)가 불법인지 여부를 나타내는 이진 레이블을 갖는다.

금융 범죄 패턴. 그림 1은 자금세탁을 나타내는 부분그래프 패턴의 일부를 보여준다(Granados and Vargas 2022; He et al. 2021; Suzumura 2022; Weber et al. 2018; Starnini et al. 2021). 안타깝게도 이들은 다소 일반적인 패턴으로, 완전히 무고한 거래들 사이에서도 광범위하게 나타난다. 결과적으로 금융 범죄 탐지는 개별 패턴의 탐지뿐만 아니라 관련된 조합의 학습에도 의존한다. 이로 인해 신경망이 이 과제에 유망한 후보가 된다. 그러나 표준 메시지 전달 GNN은 일반적으로 degree-in을 제외하고는 묘사된 패턴들을 탐지하는 데 실패한다. 다음 섹션에서는 GNN이 이러한 각 패턴을 탐지할 수 있게 하는 아키텍처 적응 기법들을 설명한다.

부분그래프 탐지. 부분그래프 패턴 H 가 주어졌을 때, 노드에 대한 부분그래프 탐지를 그래프의 각 노드에 대해 그 노드가 H 와 동형인 부분그래프의 일부인지를 결정하는 문제로 정의한다. 즉, 노드 $v \in V(G)$ 가 주어졌을 때, $E(G') \subseteq E(G)$ 이고 $V(G') \subseteq V(G)$ 이며 $v \in V(G')$ 이고 G' 이 H 와 동형(isomorphic)인 그래프 G' 가 존재하는지를 결정한다.

3.2 메시지 전달 신경망

메시지 전달 GNN은 흔히 메시지 전달 신경망(Message Passing Neural Networks, MPNN)이라 불리며, GNN의 가장 대표적인 계열을 이룬다. 여기에는 GCN(Kipf and Welling 2016), GIN(Xu et al. 2018), GAT(Velickovic et al. 2017), GraphSAGE(Hamilton, Ying, and Leskovec 2017) 등 많은 아키텍처가 포함된다. 이들은 세 단계로 작동한다. (1) 각 노드가 현재 상태 $h(v)$ 를 담은 메시지를 이웃에게 보낸다. (2) 각 노드가 이웃으로부터 받은 모든 메시지를 임베딩 $a(v)$ 로 집계한다. (3) 각 노드가 $h(v)$ 와 $a(v)$ 에 기반하여 상태를 갱신하여 새로운 상태를 만든다. 이 3단계가 GNN의 한 층을 구성하며, 반복함으로써 그래프의 점점 더 먼 곳으로부터 정보를 수집할 수 있다. 보다

형식적으로는 다음과 같다.

$$\begin{aligned} a^{(t)}(v) &= \text{AGGREGATE}(\{ h^{(t-1)}(u) \mid u \in N(v) \}), \\ h^{(t)}(v) &= \text{UPDATE}(h^{(t-1)}(v), a^{(t)}(v)), \end{aligned}$$

여기서 $\{\cdot\}$ 는 멀티집합(multiset)을 나타내고, AGGREGATE는 순열 불변(permutation-invariant) 함수이다. AGGREGATE를 AGG로 줄여 쓰고, 가독성을 위해 멀티집합을 나타낼 때 $\{\cdot\}$ 대신 $\{\cdot\}$ 를 사용할 것이다.

방향성 그래프의 경우, 노드 u 의 들어오는 이웃과 나가는 이웃을 구별할 필요가 있다. 표준 MPNN에서는 메시지가 방향성 엣지를 따라 지정된 방향으로 전달된다. 따라서 집계 단계는 들어오는 이웃의 메시지만 고려한다.

$$a^{(t)}(v) = \text{AGG}(\{ h^{(t-1)}(u) \mid u \in N_{\text{in}}(v) \}),$$

여기서 들어오는 이웃 $N_{\text{in}}(v)$ 에 대해 집계한다. 입력 그래프의 엣지도 입력 특성을 가질 수 있다. 방향성 엣지 $e = (u, v)$ 의 입력 특성을 $h^{(0)}((u, v))$ 로 표기한다. 메시지 전달 중에 엣지 특성을 사용하는 경우, 집계 단계는 다음과 같이 된다.

$$a^{(t)}(v) = \text{AGG}(\{ (h^{(t-1)}(u), h^{(0)}((u, v))) \mid u \in N_{\text{in}}(v) \})$$

이하에서는 간결함을 위해 불필요한 경우 수식에서 엣지 특성을 생략한다.

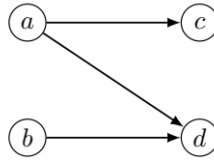


그림 2: 서로 다른 진출 차수(out-degree)를 갖는 노드(a와 b)는 방향성 메시지 전달을 사용하는 표준 MPNN으로 구별할 수 없다. 한편 순진한 양방향 메시지 전달은 노드 a와 d를 구별할 수 없다는 점에 주목하라.

4. 방법

이 섹션에서는 그림 1의 사기 패턴 탐지를 가능하게 하는 표준 MPNN(Message Passing Neural Network)을 위한 간단한 적응 기법들을 소개한다. 각 적응 기법이 탐지에 도움을 주는 패턴의 복잡성이 증가하는 순서대로 살펴본다. 각 적응 기법의 동기를 부여하는 이론적 결과를 제시하고, 섹션 7.1에서 합성 부분그래프 탐지 데이터셋에 대한 해당 실험을 포함하여 이론을 경험적으로 뒷받침한다.

4.1 역방향 메시지 전달 (Reverse Message Passing)

방향성 엣지를 사용하는 표준 MPNN에서는 한 노드가 나가는 이웃으로부터 어떤 메시지도 받지 못하며(그 이웃이 우연히 들어오는 이웃이기도 한 경우는 제외), 따라서 자신의 나가는 엣지를 셀 수 없다. 예를 들어, 표준 MPNN은 그림 2의 노드 a와 b를 구별할 수 없다. 나아가, 엣지를 무방향으로 취급하고 메시지가 양방향으로 이동하는 순진한 양방향 메시지 전달도 문제를 해결하지 못한다. 그 경우 노드는 들어오는 엣지와 나가는 엣지를 구별할 수 없기 때문이다. 따라서 같은 그림에서 노드 a와 d를 구별하는 데 실패할 것이다.

이 문제를 극복하려면 어떤 방식으로든 엣지의 방향을 표시해야 한다. 우리는 들어오는 엣지와 나가는 엣지에 대해 각각 별도의 메시지 전달 층을 사용할 것, 즉 역방향 메시지 전달(reverse message passing)의 추가를 제안한다. 이는 두 가지 엣지 유형을 가진 관계형 GNN을 사용하는 것과 유사하다는 점에 주목하라(Schlichtkrull et al. 2018). 보다 형식적으로, 집계와 갱신 메커니즘은 다음과 같이 된다.

$$\begin{aligned} a_{in}^{(t)}(v) &= AGG_{in}(\{h^{(t-1)}(u) \mid u \in N_{in}(v)\}), \\ a_{out}^{(t)}(v) &= AGG_{out}(\{h^{(t-1)}(u) \mid u \in N_{out}(v)\}), \\ h^{(t)}(v) &= UPDATE(h^{(t-1)}(v), a_{in}^{(t)}(v), a_{out}^{(t)}(v)), \end{aligned}$$

여기서 a_{in} 은 이제 들어오는 이웃의 집계이고 a_{out} 은 나가는 이웃의 집계이다. 이제 역방향 MP를 사용하는 메시지 전달 GNN이 degree-out을 해결할 수 있음을 증명한다.

명제 4.1. 합산(sum) 집계와 역방향 MP를 사용하는 MPNN은 degree-out을 해결할 수 있다.

명제 4.1의 증명은 부록 B에서 찾을 수 있다. 섹션 7.1에서는 합성 패턴 탐지 과제를 사용하여 이론이 실제로 이어지는지 확인한다.

4.2 방향성 멀티그래프 포트 번호 (Port Numbering)

사람들은 종종 같은 계좌로 여러 번 거래를 한다. 거래 네트워크에서 이들은 평행 엣지(parallel edge)로 표현된다. fan-in(또는 fan-out) 패턴을 탐지하려면 모델이 같은 이웃으로부터의 엣지와 다른 이웃으로부터의 엣지를 구별할 수 있어야 한다. 고유한 계좌 번호(또는 일반적으로 노드 ID)를 사용하면 자연스럽게 이것이 가능할 것이다. 그러나 계좌 번호를 사용하는 것은 일반화가 잘 되지 않는다. 학습 중에 모델이 사기 패턴을 식별하는 법을 학습하지 않고 사기성 계좌 번호를 암기할 수 있으며, 이는 보지 못한 계좌로 일반화되지 않는다.

대신 우리는 포트 번호(port numbering)(Sato, Yamada, and Kashima 2019)를 방향성 멀티그래프에 맞게 적응시킨다. 포트 번호는 한 노드에서 각 이웃에 로컬 ID를 할당한다. 이를 통해 노드는 연속된 메시지 전달 라운드에서 같은 이웃으로부터 오는 메시지를 식별할 수 있다. 포트 번호를 방향성 멀티그래프에 적응시키기 위해, 우리는 각 방향성 엣지에 들어오는 포트 번호와 나가는 포트 번호를 할당하고, 같은 노드로부터 오는(또는 같은 노드로 가는) 엣지는 같은 들어오는(또는 나가는) 포트 번호를 받는다. 수신 메시지에 노드의 로컬 포트 번호만 첨부하는 Sato, Yamada, and Kashima (2019)와 달리, 우리는 양방향의 포트 번호를 모두 첨부한다. 즉, 노드는 자신이 이웃에게 할당한 포트 번호와 그 이웃이 자신에게 할당한 포트 번호를 모두 본다. 이것이 우리의 표현성 논증에 결정적으로 중요하다는 것이 밝혀진다.

포트 번호는 단순 그래프에서 GNN의 표현성을 높이는 것으로 알려져 있지만, 포트 번호만을 사용하는 메시지 전달 GNN은 어떤 경우에는 3-사이클조차 탐지할 수 없다(Garg, Jegelka, and Jaakkola 2020).

일반적으로 한 노드 주변의 포트 번호 할당은 임의적이다. d 개의 들어오는 이웃을 가진 노드는 $d!$ 가지 방식으로 들어오는 포트 번호를 할당할 수 있다. 우리 데이터셋에서 이 대칭성을 깨기 위해 거래 타임스탬프를 사용하여 들어오는(또는 나가는) 이웃의 순서를 정한다. 평행 엣지의 경우, 가장 이른 타임스탬프를 사용하여 이웃의 순서를 결정한다. 타임스탬프는 금융 범죄 탐지에서 의미를 가지므로 이러한 순서 선택에는 근거가 있다. 실제로 타임스탬프가 다른 두 개의 동일한

부분그래프 패턴은 서로 다른 의미를 가질 수 있다.

이 방식으로 포트 번호를 계산하는 것은 시간이 많이 드는 단계일 수 있으며, 런타임 복잡도는 모든 엣지를 타임스탬프로 정렬하는 데 의해 지배된다. 즉 $O(m \log m)$ 이며, 여기서 $m = |E(G)|$ 이다. 그러나 포트 번호는 미리 계산할 수 있으므로 학습 및 추론 시간에는 영향이 없다. 각 엣지는 추가 엣지 특성으로서 들어오는 포트 번호와 나가는 포트 번호를 받는다. 그림 3은 포트 번호가 있는 그래프의 예를 보여준다. 이제 포트 번호를 사용하는 GNN이 fan-in과 fan-out 패턴을 올바르게 식별할 수 있음을 증명한다.

다음 증명과 포트 번호를 사용하는 이후의 증명들은 정확성을 위해 타임스탬프에 의존하지 않는다는 점에 주목하라. 그러나 포트를 고유하게 식별하는 타임스탬프를 사용할 수 있다면 GNN의 순열 불변성/동변성이 보존될 것이다.

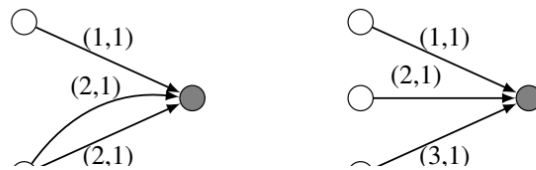


그림 3: 서로 다른 fan-in을 갖는 노드(회색)는 표준 MPNN으로 구별할 수 없다. 엣지 레이블은 각각 들어오는 포트 번호와 나가는 포트 번호를 나타낸다.

명제 4.2. 최대(max) 집계와 멀티그래프 포트 번호를 사용하는 MPNN은 fan-in을 해결할 수 있다.

증명은 부록 B에 제공된다. 역방향 MP를 추가하면 fan-out도 유사하게 해결할 수 있음을 논증할 수 있다. 두 명제 모두 섹션 7.1에서 경험적으로 확인된다.

명제 4.3. 최대 집계, 멀티그래프 포트 번호, 그리고 역방향 MP를 사용하는 MPNN은 fan-out을 해결할 수 있다.

4.3 Ego ID

역방향 MP와 멀티그래프 포트 번호는 그림 1의 일부 의심 패턴 탐지에 도움이 되지만, 방향성 사이클, 스캐터-개더 패턴, 방향성 바이클릭을 탐지하기에는 충분하지 않다. You et al. (2021)은 그래프에서 사이클 탐지를 돕기 위해 특별히 ego ID를 도입하였다. 아이디어는 "중심" 노드를 뚜렷한 (이진) 특성으로 "표시"하면, 이 노드가 일련의 메시지가 순환하여 자신에게 돌아올 때 이를 인식할 수 있고, 이로써 자신이 속한 사이클을 탐지할 수 있다는 것이다. 그러나 해당 논문의 명제 2에 대한 증명은 부정확한 것으로 밝혀졌으며, ego ID만으로는 사이클 탐지가 가능하지 않다. 부록의 그림 6에서 반례를 제시한다. 실제로 Huang et al. (2022)도 그 증명이 "워크(walk)와 경로(path)를 혼동한다"고 지적하였다.

이것이 표 1의 개별 결과에 반영되어 있음을 볼 수 있다. ego ID는 짧은 사이클 탐지에서 향상을 제공하지만, 더 긴 사이클 탐지에서는 베이스라인(GIN)에 도움이 되지 않는다. 이는 이론적으로도 설명할 수 있다. 그래프에 루프(자기 자신으로의 엣지)가 없다고 가정하면, 시작 노드로 돌아오는 길이 2와 3의 워크는 중간 노드를 반복할 가능성이 없으므로 사이클이기도 하다. 따라서 You et al.

(2021)의 명제 2가 이 경우들에 적용되며, GIN+EgoIDs가 2- 및 3-사이클 탐지에서 인상적인 F1 점수를 달성할 수 있는 것은 놀라운 일이 아니다.

그러나 역방향 MP 및 포트 번호와 결합하면, ego ID는 사이클, 스캐터-개더 패턴, 이분 부분그래프를 탐지할 수 있어 의심 패턴 목록을 완성한다. 실제로, 이러한 적응 기법을 갖춘 충분히 강력한 표준 MPNN은 임의의 두 비동형 (부분)그래프를 구별할 수 있고, 포트 번호를 일관되게 사용하면 임의의 두 동형 (부분)그래프를 잘못 구별하지 않음을 보일 수 있다. 이 두 가지 성질을 충족하는 GNN은 흔히 범용적(universal)이라 불린다. 증명의 핵심은 ego ID, 포트 번호, 역방향 MP를 사용하여 그래프의 각 노드에 고유 ID를 할당하는 방법을 보이는 것이다. 고유 노드 ID가 주어지면, 충분히 강력한 표준 MPNN은 범용적인 것으로 알려져 있다(Loukas 2019; Abboud et al. 2020).

정리 4.4. 포트 번호 및 역방향 MP와 결합된 ego ID는 연결된 방향성 멀티그래프에서 고유 노드 ID를 할당하는 데 사용될 수 있다.

증명의 아이디어는 GNN이 ego 노드의 이웃에 있는 각 노드에 고유 ID를 할당하는 레이블링 알고리즘을 어떻게 복제할 수 있는지를 보이는 것이다. 레이블링 알고리즘과 전체 증명은 부록 B에 제공된다. 이 적응 기법들의 범용성은 이 정리로부터 따라 나온다.

따름정리 4.4.1. ego ID, 포트 번호, 역방향 MP를 갖춘 GIN은 이론적으로 어떤 방향성 부분그래프 패턴이든 탐지할 수 있다.

증명은 위의 정리 4.4와 Loukas (2019)의 따름정리 3.1로부터 따라 나온다. 단순 무방향 그래프에 대해서도 유사한 진술을 할 수 있다. 역방향 MP는 방향성 엣지에서 증명이 성립하게 하는 데만 필요하므로 가정에서 제거할 수 있다.

정리 4.5. ego ID와 포트 번호는 연결된 무방향 그래프에서 고유 노드 ID를 할당하는 데 사용될 수 있다.

따름정리 4.5.1. ego ID와 포트 번호를 갖춘 GIN은 이론적으로 무방향 그래프의 어떤 부분그래프든 탐지할 수 있다.

섹션 7.1 표 1의 절제(ablation) 연구도 이론적 분석을 다시 한번 뒷받침한다. 세 가지 적응 기법의 조합이 모든 부분그래프 패턴에서 인상적인 점수를 달성한다. 엣지의 양쪽 접속 노드의 포트 번호를 모두 전달하는 것이 고유 노드 ID 추론에 결정적으로 중요하다는 점에 주목하라. 부록의 그림 5가 이를 간단한 예로 설명한다. 특히, Sato, Yamada, and Kashima (2019)가 도입한 방식의 포트 번호로는 충분하지 않다.

4.4 복잡도와 런타임

우리는 적응 기법들의 집합을 제안하므로, 최종 모델 복잡도는 기반 GNN의 선택에 따라 달라진다. 적응 기법들로 인해 발생하는 추가 런타임 비용은 부록 C에 설명한다. 전체적으로, 적응 기법들은 $O(m \log(m))$ 의 일회성 사전 계산 비용에 더해 런타임 복잡도에 상수 인자를 추가한다. GIN을

사용한 AML Small HI에서의 경험적 런타임은 부록 F.5에서 볼 수 있다.

5. 데이터셋

합성 패턴 탐지 과제. 그림 1에 나타난 AML 부분그래프 패턴을 사용하여 통제 가능한 합성 패턴 탐지 과제 테스트베드를 만든다. 핵심 설계 원칙은 원하는 부분그래프 패턴이 그래프에 사후적으로 삽입되는 것이 아니라 무작위로 나타나도록 보장하는 것이다. 패턴을 삽입하는 방식의 문제점은 무작위 분포를 왜곡하여 (노드의 차수와 같은) 단순한 지표만으로도 과제를 근사적으로 해결할 수 있다는 것이다. 예를 들어, 무작위 k -정규 그래프를 생성한 다음 패턴을 삽입하는 극단적인 경우를 생각해 보라. 패턴에 속하는 노드는 차수가 k 를 초과하는지 확인하는 것만으로 식별될 수 있다. 또한 삽입된 패턴에만 레이블이 붙는다면 무작위로 발생한 패턴은 간과될 것이다.

원하는 부분그래프 패턴이 무작위로 나타나도록 보장하기 위해, 우리는 랜덤 순환 그래프(random circulant graph) 생성기를 도입한다. 생성기의 세부 사항과 의사코드는 부록 D.1에서 찾을 수 있다. 패턴 탐지 과제에는 degree-in/out(진입/진출 엣지 수), fan-in/out(고유 진입/진출 이웃 수), 스캐터-개더, 방향성 바이클릭, 그리고 길이 6까지의 방향성 사이클이 포함된다. 자세한 설명은 부록 D.2에서 찾을 수 있다.

반자금세탁(AML). 금융 데이터에 관한 엄격한 프라이버시 규제로 인해 실제 데이터셋은 쉽게 구할 수 없다. 대신 우리는 시뮬레이션된 자금세탁 데이터(Altman et al. 2023)를 사용한다. 이 데이터셋 뒤에 있는 시뮬레이터는 가상 세계의 에이전트(은행, 기업, 개인)를 모델링하여 금융 거래 네트워크를 생성한다. 생성기는 잘 확립된 세탁 패턴을 사용하여 현실적인 자금세탁(불법) 거래를 추가한다. 우리는 소형 데이터셋 2개와 중형 데이터셋 2개를 사용하며, 각 크기별로 하나는 높은 불법 비율(HI), 다른 하나는 낮은 불법 비율(LI)을 갖는다. 데이터셋 크기와 불법 비율은 부록의 표 4에 제공된다. 60-20-20 시간적 학습-검증-테스트 분할을 사용한다. 즉, 거래를 타임스탬프로 정렬한 후 분할한다. 자세한 내용은 부록 E에서 찾을 수 있다.

이더리움 피싱 탐지(ETH). 은행이 데이터를 공개하지 않기 때문에, 실제 데이터셋을 위해 암호화폐로 눈을 돌린다. Kaggle에 공개된 이더리움 거래 네트워크(Chen et al. 2019a)를 사용하며, 여기서 일부 노드는 피싱 계좌로 레이블되어 있다. 시간적 학습-검증-테스트 분할을 사용하되, 이번에는 노드를 분할한다. 불법 계좌가 데이터셋의 후반부에 치우쳐 있기 때문에 65-15-20 분할을 사용한다. 자세한 내용과 데이터셋 통계는 부록 E에서 찾을 수 있다.

실제 방향성 그래프 데이터셋. 이론적 결과와 부분그래프 탐지 과제는 아키텍처 적응 기법의 범용적 잠재력을 보여준다. 그러나 이러한 주장을 추가로 뒷받침하려면 실제 벤치마크 데이터셋에서 모델을 테스트하는 것이 중요하다. 확립된 방향성 멀티그래프 벤치마크가 없기 때문에, 세 가지 방향성 그래프 데이터셋인 Chameleon, Squirrel(Pei et al. 2020), Arxiv-Year(Hu et al. 2020)를 가져와 이 벤치마크들의 최첨단 모델(Rusch et al. 2022)과 우리의 접근법을 비교하였다. 이 데이터셋들은 이 논문의 초점이 아니므로, 실험 세부 사항과 결과는 부록 G에 남겨둔다.

6. 실험 설정

기반 GNN과 베이스라인. 엣지 특성을 사용하는 GIN(Hu et al. 2019)을 주요 GNN 기반 모델로 사용하고 그 위에 우리의 적응 기법을 추가한다. GAT(Velickovic et al. 2017)과 PNA(Velickovic et al. 2019)도 기반 모델로 사용하며, 이들의 적응 버전을 각각 Multi-GAT, Multi-PNA라 부른다. 세

모델 모두 베이스라인으로도 고려된다. 추가로, ego ID를 갖춘 GIN은 ID-GNN(You et al. 2021) 베이스라인으로, 포트 번호를 갖춘 GIN은 CPNGNN(Sato, Yamada, and Kashima 2019) 베이스라인으로 간주될 수 있다. AML은 엣지 분류 문제이므로, 엣지 업데이트(Battaglia et al. 2018)를 사용하는 베이스라인도 포함하며 이를 GIN+EU로 표기한다. 이 접근법은 엣지를 노드로 대체하고 해당 라인 그래프(line graph)에서 GNN을 실행하는 것과 유사한데, 이는 최근 자기지도 자금세탁 탐지에서 최첨단(SOTA) 결과를 달성하였다(Cardoso, Saleiro, and Bizarro 2022). 또한 R-GCN(Schlichtkrull et al. 2018)을 베이스라인으로 포함한다. 더 광범위한 GNN 베이스라인을 포함하는 데 초점을 두지 않는 이유는 간단하다. (제안된) 적응 기법 없이는 이들이 방향성 멀티그래프를 다룰 수 있게 갖추어져 있지 않기 때문이다. 다만 "더 표현력 있는" GNN을 사용한 추가 결과는 부록 H.1에서 찾을 수 있다. 우리가 아는 한, 방향성 멀티그래프에서 SOTA 결과를 기대할 수 있는 다른 GNN은 없다.

사전 계산된 그래프 기반 특성(GF)과 트리 기반 분류기를 사용하여 노드나 엣지를 개별적으로 분류하는 금융 범죄 탐지의 평행한 연구 계열을 대표하는 베이스라인도 포함한다. 원래의 원시 특성과 추가 그래프 기반 특성을 결합하여 개별 엣지(또는 노드)에 대해 XGBoost(Chen and Guestrin 2016)와 LightGBM(Ke et al. 2017) 모델을 학습한다. 이 접근법은 금융 응용에서 SOTA 결과를 만들어 왔다(Weber et al. 2019; Lo, Layeghy, and Portmann 2022).

AML 및 ETH 데이터셋의 크기를 고려하여, 모든 GNN 기반 모델에 이웃 샘플링(Hamilton, Ying, and Leskovec 2017)을 사용한다. 각 데이터셋의 실험 설정에 대한 자세한 내용은 부록 F에서 찾을 수 있다.

채점. 데이터셋이 매우 불균형하므로 정확도(accuracy) 및 기타 일반적인 지표는 적합하지 않다. 대신 소수 클래스 F1 점수를 사용한다. 이는 은행과 규제 기관이 실제 시나리오에서 사용하는 것과 잘 부합한다.

7. 결과

7.1 합성 패턴 탐지 결과

합성 패턴 탐지 결과는 표 1에서 볼 수 있다. degree-out 결과는 표준 메시지 전달 GNN이 degree-out 과제를 해결할 수 없어 F1 점수가 44% 미만임을 보여준다. 그러나 역방향 MP를 갖춘 모든 GNN은 98% 이상의 점수를 기록하여 명제 4.1을 뒷받침한다. 다음 열은 포트 번호가 fan-in을 해결하는 데 결정적인 적응 기법임을 보여주며, 베이스라인 GIN조차 F1 점수가 상당히 높다. 반면 fan-out 과제에서는 역방향 MP와 포트 번호의 조합이 있어야 99% 이상의 점수를 얻을 수 있다. 이 결과들은 다시 명제 4.2와 4.3을 뒷받침한다. GIN 위에 적응 기법을 누적한 절제 연구도 따름정리 4.4.1을 뒷받침한다. 역방향 MP, 포트 번호, ego ID의 조합은 모든 하위 과제에서 높은 점수를 기록하며, 6-사이클 탐지만 90% 미만이다. 다른 기반 GNN 모델을 사용해도 유사한 결과를 보이며, Multi-PNA가 전체적으로 가장 좋은 결과를 달성하였다. 더욱이, 더 복잡한 과제(방향성 사이클, 스캐터-개더, 바이클릭 탐지)에서는 세 가지의 조합이 F1 점수의 첫 유의미한 향상을 이끈다. 가장 극단적인 경우인 스캐터-개더 탐지에서는 역방향 MP와 포트 번호만 사용했을 때 67.84%이던 소수 클래스 F1 점수가 ego ID를 추가하면 97.42%로 급등한다. 어떤 단일 적응 기법도 이 점수에 근접하지 못하므로 조합이 필요하다는 것이 분명하다. 방향성 4-, 5-, 6-사이클 및 바이클릭 탐지에서도 유사한 도약을 볼 수 있다. 데이터셋 크기를 늘리고 과제를 "복잡한" 하위 과제로만

제한하면 점수가 더욱 향상되어 6-사이클 탐지도 97% 이상에 도달한다. 자세한 내용은 부록 H.2에서 찾을 수 있다.

추가 절제 실험도 부록에서 찾을 수 있다. 특히, 랜덤 고유 노드 ID를 입력 특성으로 사용하여 실험을 다시 실행했는데, 노드 ID가 실제로는 포트 번호와 ego ID를 대체할 수 없음을 확인하였다.

Model	deg-in	deg-out	fan-in	fan-out	C2	C3	C4	C5	C6	S-G	B-C
GIN (Xu et al. 2018; Hu et al. 2019)	99.77	43.58	95.57	35.91	34.67	58.00	50.80	43.12	48.59	69.31	63.12
GAT (Veličković et al. 2017)	10.33	10.53	9.69	0.00	0.00	0.00	25.86	0.00	0.00	0.00	0.00
PNA (Veličković et al. 2019)	99.63	43.02	95.00	38.93	25.77	54.75	51.92	48.79	48.40	65.88	65.51
GIN+EU (Battaglia et al. 2018)	99.30	42.74	95.70	39.13	32.58	55.91	54.65	47.62	49.68	68.54	64.64
GIN+EgoIDs (You et al. 2021)	99.78	51.48	95.06	49.24	98.13	97.97	53.12	44.37	45.42	66.44	63.90
GIN+Ports (Sato, Yamada, and Kashima 2019)	99.47	45.00	99.59	41.51	27.79	56.11	42.68	41.11	44.99	67.99	65.76
GIN+ReverseMP (Jaume et al. 2019)	98.87	99.08	94.99	95.25	35.96	63.85	69.09	67.44	71.23	65.83	66.18
+Ports	98.41	98.35	98.51	99.16	39.15	63.58	69.00	70.35	75.04	67.84	65.78
+EgoIDs (Multi-GIN)	99.48	99.09	99.62	99.32	98.97	98.73	97.46	91.60	84.23	97.42	94.33

표 1: 합성 부분그래프 탐지 과제의 소수 클래스 F1 점수(%). 위에서부터 표준 MPNN 베이스라인, GIN 위에 각 적응 기법을 개별적으로 추가한 결과, GIN에 적응 기법을 누적으로 추가한 결과, 그리고 세 가지 적응 기법을 모두 적용한 다른 GNN 베이스라인(Multi-GNN)의 결과이다. Ck는 방향성 k-사이클 탐지, S-G는 스캐터-개더, B-C는 바이클릭 탐지를 나타낸다. 다섯 번의 실행에 대한 평균 소수 클래스 F1 점수를 보고한다. 가독성을 위해 표준편차는 생략한다.

7.2 AML 결과

AML 데이터셋의 결과는 표 2에서 볼 수 있다. AML Small HI에서 우리의 적응 기법은 GIN의 소수 클래스 F1 점수를 28.7%에서 57.2%로, 거의 30% 향상시킨다. 가장 큰 개선은 역방향 MP와 포트 번호에서 비롯되어 F1 점수를 28.7%에서 56.9%로 끌어올리며, ego ID는 여기서 큰 차이를 만들지 않는다. 다른 AML 데이터셋의 결과도 유사한 경향을 보이며 GIN의 전체 향상 폭은 각각 14.2%, 14.0%, 10.7%이고, 적응 기법이 추가될수록 수확 체감이 나타난다. 포트 번호에 해당하는 두 행(GIN+Ports와 +Ports)은 포트 번호를 단독으로 사용할 때와 역방향 MP 위에 추가할 때 모두 명확한 향상을 보여준다. ego ID에 대한 지지는 덜 명확하여, 개별 적응 기법으로 사용할 때는 명확한 향상이 있지만 역방향 MP와 포트 번호 위에 추가할 때는 유의미한 향상이 없다. GNN 층을 두 개만 사용했으므로, 층을 더 추가하고 더 긴 사이클도 탐지할 수 있게 되면 이 결론이 바뀔 수 있다는 점에 주목하라. 전체 적응 기법 집합은 다른 세 가지 기반 모델, 즉 GIN+EU(엣지 업데이트를 사용하는 GIN), PNA, PNA+EU로도 테스트하였다. 각 경우에, 그리고 거의 모든 AML 데이터셋에서 적응 기법 사용으로 인한 명확한 향상을 확인하였다. 이러한 향상은 이 접근법의 효과와 다용성을 강조한다. 나아가 Multi-PNA+EU가 모든 AML 데이터셋에서 모든 베이스라인을 능가함을 주목한다. 이는 그래프 기반 특성을 사용하는 트리 기반 방법(XGBoost+GFs 및 LightGBM+GFs)과 비교할 때 특히 인상적인데, 이 수작업 특성들은 시뮬레이터가 사용한 불법 자금세탁 패턴과 완벽하게 일치하기 때문이다. 더욱이 이 트리 기반 방법들은 이전 금융 응용에서 SOTA였다(Weber et al. 2019; Lo, Layeghy, and Portmann 2022).

개별 자금세탁 패턴에 대한 재현율(recall) 점수는 부록 I에서 찾을 수 있다. 자금세탁 패턴에 속하는 불법 거래의 대부분이 식별되며, 전체 데이터셋 점수는 데이터셋 내 단독(자금세탁 패턴에 속하지 않는) 불법 거래의 비율에 크게 영향을 받는다는 점에 주목할 만하다. 단독 불법 거래는 식별하기가 매우 어렵다.

GIN 기반 모델의 학습 시간과 추론 처리량은 부록의 표 5를 참조하라. 특히, 모든 적응 기법을 적용한 Multi-GIN도 단일 GPU에서 초당 18,000건 이상의 거래 추론 속도를 유지한다.

7.3 ETH 결과

마지막으로, 실제 금융 범죄 데이터셋인 이더리움 피싱 계좌 분류에서 적응 기법을 테스트한다. 결과는 표 2에 제공된다. AML 데이터셋과 유사하게, 적응 기법을 추가할수록 최종 점수가 일관되게 향상된다. 전체적으로 소수 클래스 F1 점수는 적응 기법이 없을 때 26.9%에서 역방향 MP, 포트 번호, ego ID를 적용하면 42.9%로 도약한다. 다시 가장 큰 단일 개선은 역방향 MP에서 비롯된다. 이 경우 Multi-GIN이 모든 베이스라인을 능가하지는 않지만, 적응 기법은 PNA 성능도 크게 향상시켜 Multi-PNA와 Multi-PNA+EU가 모든 베이스라인을 12% 이상 앞선다.

Model	AML Small HI	AML Small LI	AML Medium HI	AML Medium LI	ETH
LightGBM+GFs (Altman et al. 2023)	62.86 ± 0.25	20.83 ± 1.50	59.48 ± 0.15	20.85 ± 0.38	53.20 ± 0.60
XGBoost+GFs (Altman et al. 2023)	63.23 ± 0.17	27.30 ± 0.33	65.70 ± 0.26	28.16 ± 0.14	49.40 ± 0.54
GIN (Xu et al. 2018; Hu et al. 2019)	28.70 ± 1.13	7.90 ± 2.78	42.20 ± 0.44	3.86 ± 3.62	26.92 ± 7.52
PNA (Velickovic et al. 2019)	56.77 ± 2.41	14.85 ± 1.46	59.71 ± 1.91	27.73 ± 1.65	51.49 ± 4.26
GIN+EU (Battaglia et al. 2018)	47.73 ± 7.86	20.62 ± 2.41	49.26 ± 4.02	6.19 ± 8.32	33.92 ± 7.34
R-GCN (Schlichtkrull et al. 2018)	41.78 ± 0.48	7.43 ± 0.38	OOM	OOM	OOM
GIN+EgoIDs (You et al. 2021)	39.65 ± 4.73	14.98 ± 2.66	45.26 ± 2.16	11.17 ± 6.41	26.01 ± 2.27
GIN+Ports (Sato, Yamada, and Kashima 2019)	54.85 ± 0.89	21.41 ± 2.40	54.22 ± 1.94	10.51 ± 12.82	32.96 ± 0.25
GIN+ReverseMP (Jaume et al. 2019)	46.79 ± 4.97	15.98 ± 4.39	51.93 ± 2.90	14.00 ± 9.34	36.86 ± 8.12
+Ports	56.85 ± 2.64	23.80 ± 4.07	57.15 ± 0.76	11.39 ± 8.36	42.51 ± 7.16
+EgoIDs (Multi-GIN)	57.15 ± 4.99	22.12 ± 2.88	56.23 ± 1.51	14.55 ± 2.91	42.86 ± 2.53
Multi-GIN+EU	64.79 ± 1.22	26.88 ± 6.63	58.92 ± 1.83	16.30 ± 4.73	48.37 ± 6.62
Multi-PNA	64.59 ± 3.60	30.65 ± 2.00	65.67 ± 2.66	33.23 ± 1.31	65.28 ± 2.89
Multi-PNA+EU	68.16 ± 2.65	33.07 ± 2.63	66.48 ± 1.63	36.07 ± 1.17	66.58 ± 1.60

표 2: AML 및 ETH 과제의 소수 클래스 F1 점수(%). HI는 높은 불법 비율, LI는 낮은 불법 비율을 나타낸다. 모델은 표 1과 같이 구성되어 있다. "OOM"은 모델이 GPU 메모리 부족에 도달했음을 나타낸다.

8. 결론

이 연구는 기존의 메시지 전달 GNN을 증명 가능한 강력한 방향성 멀티그래프 학습기로 변환할 수 있는 일련의 직관적인 적응 기법을 탐구하였다. 그래프 신경망 분야에 대한 우리의 기여는 세 가지이다. 첫째, 이론적 분석은 서로 다른 GNN 적응/증강을 결합하는 힘에 관한 기존 문헌의 주목할 만한 공백을 다룬다. 구체적으로, 포트 번호 및 역방향 메시지 전달과 결합된 ego ID가 GIN과 같은 충분히 강력한 메시지 전달 GNN이 고유 노드 ID를 계산하고 따라서 어떤 방향성 부분그래프 패턴이든 탐지할 수 있게 한다는 것을 증명하였다. 둘째, 이론적 발견을 다양한 합성 부분그래프 탐지 과제로 경험적으로 검증하였다. 실제 결과는 이론적 기대와 거의 정확히 일치하여, 더 복잡한 부분그래프를 탐지하려면 세 가지 적응 기법 모두의 조합이 필요함을 확인하였다. 마지막으로, 적응 기법들이 두 가지 중요한 금융 범죄 문제, 즉 자금세탁 거래 탐지와 피싱 계좌 탐지에 어떻게 적용될 수 있는지를 보였다. 제안된 적응 기법으로 향상된 GNN은 두 과제 모두에서 관련 베이스라인과 대등하거나 이를 능가하는 인상적인 결과를 달성하였다. 역방향 메시지 전달과 포트 번호가 최고 점수에 도달하는 데 다시 한번 결정적임이 입증되었으나, ego ID는 이 데이터셋들에서는 큰 추가 이점을 제공하지 않음을 발견하였다.

이 연구가 금융 범죄 응용에 초점을 맞추었지만, 이론과 실제 결과는 더 넓은 관련성을 갖는다. 가까운 미래의 연구로는 다른 방향성 멀티그래프 문제에 우리 방법의 응용을 탐구하는 것이 있을 수 있다. 세 가지 실제 데이터셋에서 유망한 결과를 보이는 초기 탐색은 부록에서 찾을 수 있다. 그러나 다양한 도메인에서의 일반적 적용 가능성을 확인하려면 추가 실험이 필요하다. 또한 향후 연구로는 서로 다른 부분그래프 탐지 문제의 계산 복잡도와 GNN 성능 간의 관계를 탐구할 수 있을 것이다.

감사의 글

이 연구에 대한 스위스 국립과학재단(Swiss National Science Foundation)의 지원(프로젝트 번호: 172610 및 212158)에 감사드린다.

참고문헌

(참고문헌 목록은 원문과 동일하게 영문 원본 그대로 유지한다. Abboud et al. 2020; Altman et al. 2023; Babai, Erdos, and Selkow 1980; Barceló et al. 2021; Battaglia et al. 2016; Battaglia et al. 2018; Blanusa et al. 2022; Blanuša, Atasu, and lenne 2023; Bongini et al. 2021; Bouritsas et al. 2022; Cardoso, Saleiro, and Bizarro 2022; Chen et al. 2019a; Chen and Guestrin 2016; Chen, Wei, and Huang 2018; Chen et al. 2020; Chen et al. 2019b; Derrow-Pinion et al. 2021; Dwivedi et al. 2021; Egressy and Wattenhofer 2022; Feng et al. 2019; Frasca et al. 2022; Garg, Jegelka, and Jaakkola 2020; Granados and Vargas 2022; Hamilton, Ying, and Leskovec 2017; He et al. 2021; Hornik, Stinchcombe, and White 1989; Hu et al. 2020; Hu et al. 2019; Huang et al. 2022; Jaume et al. 2019; Kanezashi et al. 2022; Ke et al. 2017; Keisler 2022; Kipf and Welling 2016; Li et al. 2019; Li et al. 2021; Liang et al. 2019; Lim et al. 2021; Liu et al. 2018; Lo, Layeghy, and Portmann 2022; Loukas 2019; Ma et al. 2019; Maron et al. 2019; Medvedev 2022; Morris et al. 2019; Nicholls, Kuppa, and Le-Khac 2021; Papp et al. 2021; Papp and Wattenhofer 2022; Pei et al. 2020; Rao et al. 2021; Rusch et al. 2022; Sato, Yamada, and Kashima 2019; Sato, Yamada, and Kashima 2021; Schlichtkrull et al. 2018; Shu, Wang, and Liu 2019; Starnini et al. 2021; Suzumura 2022; Tong et al. 2020; Velickovic et al. 2017; Velickovic et al. 2019; Watts and Strogatz 1998; Weber et al. 2018; Weber et al. 2019; Wu et al. 2020; Xu et al. 2021; Xu et al. 2018; Yang et al. 2021; You et al. 2021; Zhang and Li 2021; Zhang et al. 2019; Zhang et al. 2021; Zhao et al. 2021; Zhou et al. 2020.)

부록 A. 동기 부여 예시

A.1 금융 거래 그래프 예시

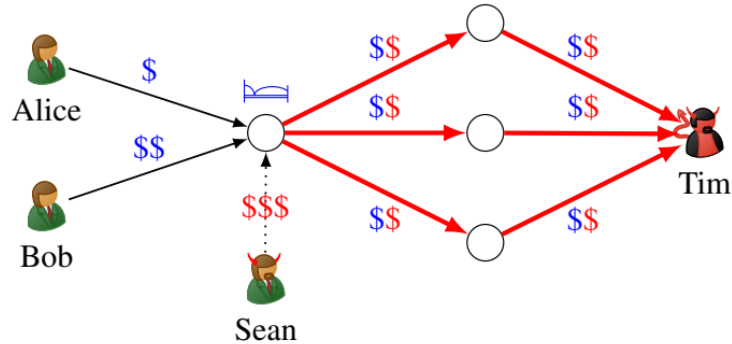


그림 4: 금융 거래 네트워크에서의 자금세탁 예시. Alice와 Bob은 Tim이 이끄는 범죄 조직이 운영하는 호텔에 투숙한다. Sean은 범죄 활동으로 얻은 검은돈(빨간 달러)을 Tim에게 전달하려 한다. 이들은 돈을 세탁하기 위해 호텔을 이용한다. 검은 현금을 투숙객의 깨끗한 돈과 섞고, 여러 계약업체에 물품 대금을 지급한 다음, 그 지급금을 Tim에게 이체한다. Sean에서 호텔로의 자금 이체는 은행과 금융 당국으로부터 숨겨진 현금 지급이다(점선 엣지). 그러나 스캐터-개더 패턴(굵은 빨간 엣지)이 자금세탁 계획을 드러낼 수 있다.

A.2 양방향으로 포트 번호 전달하기

Sato, Yamada, and Kashima (2019)가 도입한 표준 포트 번호는 고유 노드 ID를 할당하기에 충분하지 않다는 점에 주목하라. Sato, Yamada, and Kashima (2019)는 노드의 로컬 포트 번호만 수신 메시지에 첨부한다. 이와 대조적으로, 우리는 양방향의 포트 번호를 모두 첨부한다. 즉, 노드는 자신이 이웃에게 할당한 포트 번호와 그 이웃이 자신에게 할당한 포트 번호를 모두 본다. 이것이 고유 노드 ID 추론에 결정적으로 중요하다는 것이 밝혀진다. 예를 들어, 그림 5와 같이 하나의 내부 노드와 3개의 외 노드로 구성된 단순 무방향 스타(star) 그래프를 생각해 보라. 내부 노드는 외 노드들에 고유한 포트 번호를 할당하고, (Sato, Yamada, and Kashima 2019)에 따라 이 포트 번호 각각을 세 외 노드로부터의 해당 수신 메시지와 연결(concatenate)하여, 외 노드들을 구별하고 메시지를 귀속시킬 수 있다. 그러나 내부 노드의 포트 번호는 발신 메시지에 첨부되지 않으며, 외 노드는 하나의 포트(이웃)만 가지므로 매 메시지 전달 라운드에서 동일한 포트 번호가 첨부된 동일한 메시지를 받는다. 따라서 내부 노드가 ego ID를 가진 중심 노드일 때 외 노드들은 고유 ID를 얻을 방법이 없다. 마찬가지로, 세 번째 외 노드가 중심 노드일 때 표시되지 않은 두 노드는 고유 노드 ID를 얻을 방법이 없다. 우리의 접근법은 포트 번호를 양방향으로 메시지에 첨부함으로써 이 문제를 해결한다.

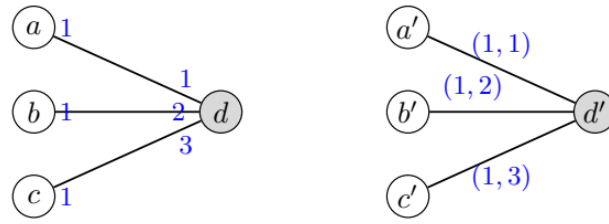


그림 5: Sato, Yamada, and Kashima (2019)는 왼쪽에 표시된 것처럼 로컬 포트 번호만 수신 메시지에 첨부한다. 포트 번호는 파란색으로 표시된다. 예를 들어, 노드 c는 d로부터의 모든 메시지에 포트 번호 1을 첨부한다. 따라서 특히 노드 a, b, c는 결코 구별될 수 없다. 우리는 엣지 양쪽의 포트 번호를 엣지 특성으로 사용하므로, 양쪽 포트 번호를 두 접속 노드가 모두 볼 수 있다. 이 예시에서는 문제가 되지 않을 수 있다(a, b, c는 같은 정답 레이블을 가질 가능성이 높다). 그러나 이는 고유 노드 ID를 생성/전파할 수 없음을 의미한다.

A.3 Ego ID 사이클 탐지 반례

ego ID만으로는(You et al. 2021) 6-사이클을 탐지하기에 충분하지 않음을 보이는 반례를 제시한다. 그림 6의 노드 u와 u'을 고려하라. ego ID를 사용하는 표준 MPNN에서는(u와 u'이 중심 노드) 같은 레이블을 받게 되지만, u'은 여러 5-사이클의 일부이고 u는 아니다. 직관적인 이유를 살펴보면, 먼저 파란색 엣지가 한쪽에서는 두 개의 3-사이클을, 다른 쪽에서는 하나의 6-사이클을 형성하며, 1-WL 테스트만으로는 이들을 구별할 수 없다는 점에 주목하라. 이제 u와 u'이 중심 노드일 때, 추가 정보는 u와 u'이 표시되었다는 것뿐이지만, 두 노드는 자신의 그래프에서 완전히 연결되어 있으므로 이 표시가 다른 노드들을 구별하는 데 도움이 되지 않고, 이웃들은 구별 불가능한 상태로 남아 실제로 두 ego 그래프는 구별 불가능하게 된다. 이는 5-사이클에 의존하는 어떤 과제에서도 두 노드가 모두 올바르게 레이블될 수 없음을 의미한다. 다른 각 노드 쌍(예: a와 a')에 대한 예측은 동일하지 않다는 점에 주목하라. 이 노드들이 ego ID를 가진 중심 노드일 때, GNN은 왼쪽의 노드들은 3-사이클에 있고 오른쪽의 노드들은 아님을 탐지할 수 있기 때문이다.

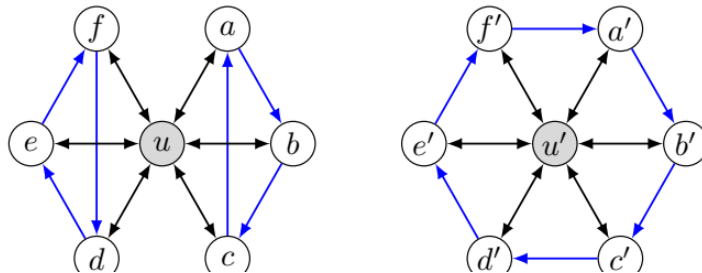


그림 6: 서로 다른 5-사이클 개수를 갖지만 ego ID를 사용하는 표준 MPNN으로 구별할 수 없는 예시 노드(u와 u'). 예를 들어 (u', a', b', c', d', u')은 오른쪽의 5-사이클이지만, 왼쪽에는 u가 포함된 5-사이클이 없다. 이 예시는 무방향 엣지에서도 성립한다.

부록 B. 증명

B.1 명제 4.1의 증명

증명. 합산 집계를 사용하는 메시지 전달 GNN을 고려하라. 그러면 단일 GNN 층과 모든 $v \in V(G)$ 에 대한 초기 노드 특성 $h^{(0)}(v) = 1$ 로, $a_{in}^{(t)}(v) = d_{in}(v)$ 이고 $a_{out}^{(t)}(v) = d_{out}(v)$ 이다. $UPDATE(h, a_{in}, a_{out}) = a_{out}$ 으로 선택하면 $h^{(1)}(v) = a_{out}^{(t)}(v) = d_{out}(v)$ 를 얻는다. $\square$

B.2 명제 4.2의 증명

증명. 들어오는 포트의 번호에 연속된 정수가 사용되므로, 최대 집계 함수는 들어오는 포트 번호의 최댓값을 직접 출력하여 원하는 결과를 얻을 수 있다. $\square$

이 명제는 합산 집계를 사용하는 MPNN에 대해서도 성립한다. 그러나 그 증명은 덜 직관적이며 멀티집합에 대한 합산 집계의 범용성(Xu et al. (2018)의 보조정리 5)에 의존한다.

B.3 정리 4.4의 증명

이 증명의 목적을 위해, 다층 퍼셉트론(MLP)에 대한 보편 근사 정리(Universal Approximation Theorem)(Hornik, Stinchcombe, and White 1989)를 가정한다. 즉, 유한 지지집합을 가진 임의의 연속 함수는 적어도 하나의 은닉층을 가진 MLP로 근사될 수 있다. GNN 층에 대한 명시적 구성을 피하기 위해 이 결과를 참조한다. 또한 GNN이 최소(min) 집계를 사용한다고 가정한다. 이는 들어오는 메시지 집합에 MLP를 원소별로 적용한 후 최솟값을 취하는 방식으로 구성된다.

증명. 처음에, ego 노드 또는 루트 노드는 노드 특성 $h^{(0)}(r) = 1$ 로 시작하며, 이것이 최종 노드 ID가 된다. 다른 노드 $v \in V(G) \setminus \{r\}$ 는 $h^{(0)}(v) = 0$ 으로 시작하며, 이는 아직 최종 ID가 할당되지 않았음을 나타낸다. 노드 ID는 층별로 할당되어, r 에서 k 홑 떨어진 노드는 k 번의 메시지 전달 라운드 후에 최종 ID를 받는다.

이전 라운드에서 활성 노드로부터 메시지를 받은 노드를 활성(active) 노드라 부른다. $t = 0$ 일 때 루트 노드만 활성 상태이다. 노드가 활성화되면 남은 모든 라운드에서 비활성으로 전환된다. GNN은 은닉 노드 임베딩의 한 차원을 사용하여 노드가 현재 활성인지 나타내고 다른 차원으로 노드가 이미 활성이었던지를 나타내어 활성 노드를 추적할 수 있다. 이 정보는 메시지에 "포함"되어 이웃 노드가 언제 활성화할지 알게 된다. 루트 노드로부터 정확히 거리 k (엣지는 어느 방향으로든 통과 가능)에 있는 노드만 k 번째 라운드에서 활성화된다는 점에 주목하라.

비활성 노드의 메시지를 무시함으로써, GNN은 알고리즘 1을 복제할 수 있다. 각 라운드에서 노드는 이웃으로부터 메시지를 받고, 자신의 ID와 들어오는(또는 나가는) 포트 번호로 구성된 메시지를 각 들어오는(또는 나가는) 이웃에게 보낸다. 메시지는 노드 ID와 포트 번호를 연결하여 만든 $2n$ 진수의 정수이다(들어오는 엣지인 경우 포트 번호에 n 이 더해진다). 메시지는 노드 ID 제안으로 볼 수 있으며, 모든 활성 노드가 각 이웃에게 노드 ID를 제안한다. 그러면 제안을 받은 모든 노드는 이 제안 중 가장 작은 것을(역시 $2n$ 진수로) 자신의 노드 ID로 받아들이고 활성화된다.

GNN에서는 이것이 수신자의 관점에서 복제될 수 있다. 각 라운드에서 노드는 이웃 노드의 현재 상태와 엣지를 따라 있는 포트 번호로 구성된 메시지를 이웃으로부터 받는다. 각 노드 v 의 MLP는 비활성 노드의 메시지를 "무시"한다. 예를 들어 이를 매우 큰 값으로 인코딩한다. 나머지 메시지에 대해 v 는 이웃의 현재 상태와 포트 번호를 이웃의 노드 ID(활성 여부를 나타내는 두 차원을 뺀 현재

상태)와 v 가 그 이웃에게 부여한 포트 번호의 연결로 인코딩한다. 이런 방식으로 MLP는 본질적으로 알고리즘의 제안을 구성한다. 그런 다음 노드 v 는 최소 제안을 선택한다.

Algorithm 1: BFS node ID assignment (Theorem 4.4)

Input: Connected directed multigraph $G = (V, E)$ with n nodes, with diameter D and with root (or ego) node $r \in V$. Active nodes $X \subseteq V$ and finished nodes $F \subseteq V$. Port numbering $P : E \rightarrow \mathbb{N}^2$.

Output: Unique node IDs $h(v)$ for all $v \in V$ (in base $2n$)

```

1:  $h(r) \leftarrow 1$ ;  $h(v) \leftarrow 0$  for all  $v \in V \setminus \{r\}$ 
2:  $F \leftarrow \emptyset$ ;  $X \leftarrow \{r\}$ 
3: for  $k \leftarrow 1$  to  $D$  do
4:   for  $v \in V$  do
5:     if  $v \in X$  then
6:       send  $h(v) \parallel P((v, u))_{out}$  to  $u \in N_{out}(v)$ 
7:       send  $h(v) \parallel n + P((u, v))_{in}$  to  $u \in N_{in}(v)$ 
8:        $F \leftarrow F \cup \{v\}$ ;  $X \leftarrow X \setminus \{v\}$ 
9:     if  $v \notin F$  then
10:      if Incoming messages  $M(v) \neq \emptyset$  then
11:         $h(v) \leftarrow \min\{M(v)\}$ 
12:         $X \leftarrow X \cup \{v\}$ 

```

알고리즘 1: BFS 노드 ID 할당 (정리 4.4)

입력 그래프가 연결되어 있으므로, D 가 그래프의 지름이기 때문에 그래프의 모든 노드가 D 라운드 이내에 도달된다(제안을 받는다). 따라서 모든 노드가 노드 ID를 갖게 된다.

이 노드 ID가 고유함을 보이는 것이 남아 있다. 먼저, 루트로부터 거리가 다른 노드는 같은 노드 ID로 끝날 수 없다는 점에 주목하라. 거리 k 의 노드는 k 번째 라운드에서 첫 제안을 받으므로 정확히 $k + 1$ 자리의 ID를 갖게 된다. 마지막으로, 귀납적 논증으로 활성 노드(같은 거리의 노드)는 같은 노드 ID를 가질 수 없음을 보인다. $X = \{r\}$ 인 시작 시점에는 분명히 참이다. 이제 이전 라운드($k - 1$)의 모든 활성 노드가 서로 다른 노드 ID를 가졌다고 가정하면, (k 번째 라운드의) 두 활성 노드가 같은 ID를 가지는 유일한 방법은 같은 이웃 노드의 제안을 받아들이는 경우이다. 귀납 가정하에서 서로 다른 노드의 제안은 이미 첫 $k - 1$ 자리에서 다르기 때문이다. 그러나 두 활성 노드가 같은 노드의 제안을 받아들였다면, 이들은 서로 다른 포트 번호를 받았을 것이다. 들어오는 포트는 서로 구별되고, 나가는 포트도 서로 구별되며, 모든 들어오는 포트에는 n 이 더해져 어떤 나가는 포트와도 같을 수 없다. 따라서 활성 노드는 항상 고유한 제안을 받아들인다. $\square$

증명에 사용된 알고리즘은 알고리즘 1에 주어져 있다. 그림 7은 이 알고리즘의 출력 예시를 보여준다.

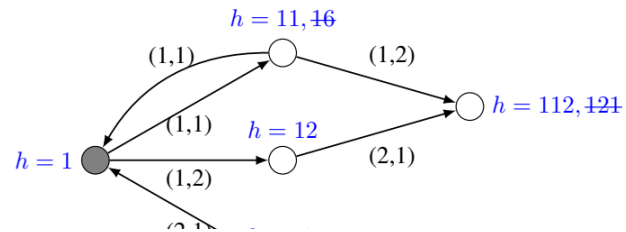


그림 7: 루트(또는 ego) 노드가 회색으로 표시된 알고리즘 1의 출력 예시. 엣지 레이블은 (들어오는, 나가는) 방향성 멀티그래프 포트 번호를 보여준다. 파란 숫자는 노드 ID 제안을 나타내며, 거절된 제안은 취소선이 그어져 최종 할당된 노드 ID만 남는다. 이 예시에서 $n = 5$ 이므로 노드 ID는 10진수로 이해하면 된다.

부록 C. 복잡도와 런타임

우리는 적응 기법들의 집합을 제안하므로, 모델의 최종 시간 복잡도는 기반 GNN의 선택에 따라 달라진다. 아래에서 적응 기법들로 인해 발생하는 추가 런타임 비용을 설명한다. GIN을 사용한 경험적 런타임은 부록 표 5에서 볼 수 있다.

역방향 메시지 전달은 시간 복잡도에 상수 인자를 곱한다. 이론적으로 이 상수 인자는 2이지만, 실제로는 모델의 메모리 요구량도 2배로 증가하므로 배치 크기를 줄여야 하고 더 큰 상수 인자로 이어진다(표 5 참조).

포트 번호는 단순히 입력 엣지 특성의 크기를 증가시킬 뿐이므로 학습 및 추론의 시간 복잡도를 증가시키지 않는다. 이는 표 5의 경험적 런타임으로 확인된다. 그러나 타임스탬프 기반 포트 번호 계산은 $O(m \log(m))$ 복잡도의 사전 계산 단계를 추가하며, 이는 학습 전에 한 번에 수행할 수 있다.

ego ID도 표 5에서 볼 수 있듯이 런타임 복잡도를 증가시키지 않는다. 전체적으로, 적응 기법들은 $O(m \log(m))$ 복잡도의 사전 계산 런타임에 더해 런타임 복잡도에 상수 인자를 추가한다.

부록 D. 합성 패턴 탐지 데이터셋

원하는 부분그래프 패턴이 무작위로 나타나도록 보장하기 위해, 우리는 랜덤 순환 그래프 생성기를 도입한다. 생성기는 노드 수 n , 평균 차수 d , 그리고 생성된 그래프의 로컬 밀도를 제어하는 반경 r 로 매개변수화된다. 이 매개변수들이 함께 다양한 부분그래프 패턴의 생성 비율을 제어한다. 예를 들어, 높은 평균 차수와 낮은 반경을 선택하면 더 밀집된 패턴도 무작위로 나타나게 할 수 있다.

D.1 랜덤 순환 그래프

랜덤 순환 그래프 $G_{n,d,r}$ 은 n 개의 노드 $\{1, 2, \dots, n\}$ 과 $\text{floor}(nd/2)$ 개의 엣지를 갖는다. 각 엣지의 한쪽 끝 u 는 $\{1, 2, \dots, n\}$ 에서 균일하게 샘플링되고, 다른 쪽 끝 v 는 u 를 중심으로 표준편차 r 인 정규분포에서 샘플링된 후 가장 가까운 정수로 반올림된다. 즉, $v = \text{floor}(X + 1/2)$ 이며 여기서 $X \sim N(u, r)$ 이다. 알고리즘 2는 이 알고리즘의 단계별 설명을 보여준다.

Algorithm 2: Random Circulant Graph Generator

Parameters: Number of nodes n , average degree d , locality radius r .

Output: A graph $G = (V, E)$.

- 1: $N_{edges} \leftarrow \lfloor \frac{nd}{2} \rfloor$
- 2: $\vec{u} \leftarrow (u_i \mid i = 1, \dots, N_{edges}) \stackrel{\text{iid}}{\sim} \text{unif}(\{1, \dots, n\})$ ▷ Sample tail nodes
- 3: $\vec{v} \leftarrow (v_i \mid i = 1, \dots, N_{edges}) \stackrel{\text{iid}}{\sim} \mathcal{N}(u_i, r)$ ▷ Sample head nodes
- 4: $\vec{v} \leftarrow (\lfloor v_i + \frac{1}{2} \rfloor \mid i = 1, \dots, N_{edges})$ ▷ Round head nodes to nearest integer
- 5: $V(G) = \{1, \dots, n\}$
- 6: $E(G) = \{(u_i, v_i) \mid i = 1, \dots, N_{edges}\}$

알고리즘 2: 랜덤 순환 그래프 생성기

이 모델은 Watts-Strogatz 스몰월드 모델(Watts and Strogatz 1998)과 유사하다. 이 모델에서도 노드가 고리 위에 배열되고 각 노드는 가장 가까운 k 개의 이웃과 연결되며, 엣지는 선택된 확률로 무작위 재배선된다. 그러나 표준 Watts-Strogatz 구현은 무방향 그래프를 생성하고 한 쌍의 정점 사이에 여러 엣지를 허용하지 않는다. 특히, 무작위 방향 부여 Watts-Strogatz 그래프(무방향 Watts-Strogatz 그래프를 생성한 다음 각 엣지의 방향을 독립적으로 균일 무작위로 선택하여 만든 방향성 그래프)는 길이 2의 방향성 사이클과 팬 패턴의 탐지를 테스트할 수 없다(평행 엣지가 없으면 팬 패턴은 차수 패턴과 동일하다).

D.2 패턴 탐지 과제

합성 패턴 탐지 데이터셋에는 다음 (하위)과제가 포함된다.

- **Degree-in (-out):** 노드의 degree-in과 -out은 각각 들어오는 거래와 나가는 거래의 수이다. 여러 거래가 같은 노드에서 올 수 있으며, 이들은 별도로 계산된다는 점에 주목하라.
- **Fan-in (-out):** 노드의 fan-in과 -out은 각각 고유한 들어오는 이웃과 나가는 이웃의 수이다. 여러 엣지를 가진 이웃은 한 번만 계산되지만, 한 노드가 들어오는 이웃이자 나가는 이웃이 될 수 있으므로 fan-in과 fan-out 계수 모두에 기여할 수 있다.
- **Cycle (k):** 노드가 길이 k 의 방향성 사이클의 일부이면 양성 레이블을 받는다. 방향성 사이클 패턴만 고려하며 "방향성"이라는 단어는 생략한다.
- **Scatter-Gather:** 노드가 적어도 2개의 중간 노드를 가진 스캐터-개더 패턴의 싱크 노드이면 양성 레이블을 받는다. 스캐터-개더 패턴은 소스 노드, 중간 노드, 싱크 노드로 구성된다. 싱크 노드 v 가 주어졌을 때, u 에서 v 로의 길이 2의 방향성 경로가 있으면 노드 u 를 소스 노드로 간주한다. u 에서 v 로 w_i 를 경유하는 길이 2의 방향성 경로가 있으면, 즉 (u, w_i) 와 (w_i, v) 가 모두 $E(G)$ 에 있으면 노드 w_i 를 중간 노드로 간주한다. 중간 노드가 하나뿐인 스캐터-개더 패턴은 단순히 길이 2의 방향성 경로라는 점에 주목하라. 이것은 우리가 관심 있는 패턴을 포착하지 못하므로, 최소 2개의 중간 노드를 요구한다.
- **Gather-Scatter:** 이 패턴은 fan-in과 fan-out의 조합이므로 하위 과제로 명시적으로 포함하지 않는다.
- **Biclique:** 바이클릭(또는 완전 이분 그래프)은 첫 번째 집합의 모든 노드가 두 번째 집합의 모든 노드와 연결된 이분 그래프이다. 우리는 첫 번째 집합의 모든 노드가 두 번째 집합의 모든 노드로 향하는 적어도 하나의 나가는 엣지를 갖는 방향성 바이클릭의 특수한 경우를 고려한다. 첫 번째 집합의 노드를 소스 노드, 두 번째 집합의 노드를 싱크 노드라 부른다. 과제는 (최소) 2개의 소스 노드와 (최소) 2개의 싱크 노드를 가진 방향성 $K_{2,2}$ 바이클릭의 싱크 노드인 모든 노드를 식별하는 것이다.

모든 (하위)과제는 하나의 전체 과제로 결합된다. 즉, 모델의 출력은 벡터이며 벡터의 각 원소는 하위 과제 중 하나에 해당한다.

우리는 원하는 패턴을 찾아 이진 노드 레이블을 생성하는 패턴 탐지 알고리즘을 구현한다. 매개변수는 상대적으로 균형 잡힌 레이블이 되도록 조정되지만, (모든 하위 과제에 대해) 완벽한 분할이 보장되지 않으므로 모델 성능 측정에 정확도 대신 소수 클래스 F1 점수에 의존한다. 최종 데이터셋의 모든 하위 과제는 최소 19%, 최대 68%의 양성 레이블을 갖는다.

이진 레이블을 생성하기 위해 경우에 따라 임계값이 사용된다. 예를 들어, degree-in 과제에서는 진입 차수가 3보다 큰 노드가 레이블 1을 받고, 낮은 진입 차수 노드는 레이블 0을 받는다. degree-in/-out과 fan-in/-out의 임계값은 3이다. degree-in-time-span과 maximum-degree-in의 임계값은 각각 1과 4이다. 패턴의 예시는 그림 1에 제공된다.

우리는 $n = 8192$ 개 노드, 평균 차수 $d = 6$, 반경 $r = 11.1$ 을 사용하여 패턴 탐지 데이터를 생성한다. 학습, 검증, 테스트 세트 간의 정보 누출을 피하기 위해 각 데이터셋에 대해 독립적인 랜덤 순환 그래프 $G_{n,d,r}$ 을 생성한다. 표 3은 각 하위 과제의 임계값과 레이블 분포를 보여준다.

Subtask	Threshold	Positive Ratio
Degree-in	>3	0.352
Degree-out	>3	0.349
Fan-in	>3	0.324
Fan-out	>3	0.323
Cycle (2)	-	0.191
Cycle (3)	-	0.344
Cycle (4)	-	0.527
Cycle (5)	-	0.677
Cycle (6)	-	0.779
Scatter-Gather	-	0.321
Biclique	-	0.318

표 3: 합성 패턴 탐지 과제 통계

부록 E. AML 및 ETH 데이터셋

E.1 AML 데이터 분할

60-20-20 시간적 학습-검증-테스트 분할을 사용한다. 즉, 거래 인덱스를 타임스탬프로 정렬한 후 분할한다. 데이터 분할은 두 개의 타임스탬프 t_1 과 t_2 로 정의된다. 학습 인덱스는 t_1 이전의 거래, 검증 인덱스는 t_1 과 t_2 사이의 거래, 테스트 인덱스는 t_2 이후의 거래에 해당한다. 그러나 검증 및 테스트 세트 거래는 패턴을 식별하기 위해 이전 거래에 접근해야 한다. 그래서 학습, 검증, 테스트 그래프를 구성한다. 이는 금융 거래 그래프를 동적 그래프로 간주하고 $t_1, t_2, t_3 = t_{\max}$ 시점에 세 개의 스냅샷을 찍는 것에 해당한다. 학습 그래프는 학습 거래(와 해당 노드)만 포함한다. 검증 그래프는 학습 및 검증 거래를 포함하지만 검증 인덱스만 평가에 사용된다. 테스트 그래프는 모든 거래를 포함하지만 테스트 인덱스만 평가에 사용된다. 이것이 새로운 거래 배치를 분류하려는 은행과 금융 당국이 직면하는 가장 가능성 높은 시나리오이다.

Dataset	# nodes	# edges	Illicit Ratio
AML Small HI	0.5M	5M	0.07%
AML Small LI	0.7M	7M	0.05%
AML Medium HI	2.1M	32M	0.11%
AML Medium LI	2.0M	31M	0.05%
ETH	2.9M	13M	0.04%

표 4: AML 및 ETH 과제 통계. HI는 높은 불법 비율, LI는 낮은 불법 비율을 나타낸다.

E.2 ETH 데이터셋

Kaggle 데이터셋은 1165개의 신고된 피싱 노드로부터 시작하여 너비 우선 탐색으로 전체 이더리움 거래 네트워크를 크롤링하여 구성되었다. 데이터셋에는 원래 네 가지 엣지 특성이 있다. 거래 시간, 거래 금액, 소스 노드 주소, 대상 노드 주소이다. 각 노드는 하나의 특성, 즉 계좌 주소를 갖는다. 이 정보는 최소한이므로, Ethereum ETL(Medvedev 2022)을 사용하여 블록체인에서 직접 가져온 추가 거래 특성으로 데이터셋을 보강한다. nonce, 블록 번호, gas, gas 가격을 추가한다. 이들은 각각 주어진 거래 이전에 발신자가 수행한 거래 수, 이 거래를 포함하는 블록, 거래가 처리되도록 발신자가 제공한 거래 수수료를 나타내는 암호화페 용어이다. 표 4는 노드(계좌)와 엣지(거래)의 수 및 불법 비율을 보여준다. AML과 달리, 이것은 노드 분류 과제로 목표는 노드를 피싱 계좌로 분류하는 것이다.

AML 사례와 달리, 피싱 노드를 식별하기 위해 어떤 부분그래프 패턴을 찾아야 할지 사전에 알지 못한다. 이를 위해 사기성 클러스터를 정의하고 시각화하여 모델이 탐지해야 할 패턴의 단서를 제공하였다. 클러스터는 1165개의 신고된 피싱 노드로부터 시작하여 1-홉 이웃을 추가한 다음 공통 노드를 공유하는 클러스터를 병합하여 구성하였다. 그림 8은 이러한 사기성 클러스터 중 하나의 예를 보여준다.

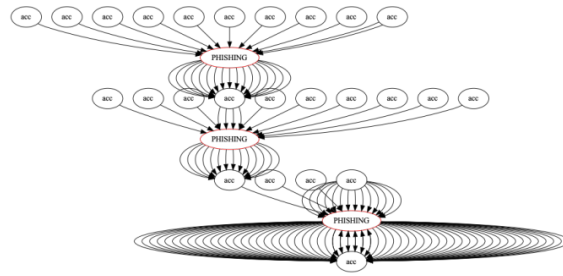


그림 8: ETH 피싱 데이터셋에서 식별된 사기성 클러스터 중 하나. 사기성 노드를 식별하는 데 도움이 될 수 있는 패턴은 대부분 1-홉, 즉 진입 차수, 진출 차수, fan-in, fan-out 패턴과 2-사이클이다.

AML과 유사하게 시간적 학습-검증-테스트 분할을 사용하지만, 이번에는 노드를 분할한다. 분할하기 전에 노드를 처음으로 관여한 거래(발신자 또는 수신자로서) 순으로 정렬한다. 이것도 다시 임계 시간 t_1 과 t_2 를 제공하며, 이 시간을 사용하여 학습, 검증, 테스트 그래프를 만든다. 자세한 내용은 AML 데이터셋 분할을 참조하라. 이번에는 65-15-20 분할을 사용하는데, 불법 계좌가 데이터셋의 후반부에 치우쳐 있고 데이터 전반부에는 불법 계좌가 거의 없기 때문이다.

부록 F. 실험 세부 사항

F.1 하이퍼파라미터 튜닝과 채점

무작위 샘플링을 사용하여 좋은 하이퍼파라미터 범위를 찾았다. 이 좁은 범위에서 두 번째 무작위 샘플링 라운드를 수행하여 최종 하이퍼파라미터 집합을 선택하였다. 다음 하이퍼파라미터를 변화시켰다. GNN 층 수, 은닉 임베딩 크기, 학습률, 드롭아웃, 그리고 (가중 손실 함수를 위한) 소수 클래스 가중치. 무작위 샘플 수는 특정 데이터셋에서 모델의 학습 시간에 따라 10에서 50 사이였다.

최종 결과를 얻기 위해, 최고의 검증 점수를 가진 하이퍼파라미터를 사용하여 다섯 개의 서로 다른 랜덤 시드로 초기화된 다섯 모델을 학습한다. 그런 다음 다섯 번의 실행 결과를 평균하고 표준편차를 계산하여 보고된 점수를 얻는다.

F.2 그래프 기반 특성을 사용하는 LightGBM

사전 계산된 그래프 특성을 사용하여 노드나 엣지를 개별적으로 분류하는 LightGBM 베이스라인을 포함한다. 복잡한 부분그래프 특성을 순진하게 계산하는 것은 엄청나게 비쌀 수 있으므로, 부분그래프 패턴을 열거하기 위해 효율적인 구현(Blanuša, Atasu, and lenne 2023; Blanus, lenne, and Atasu 2022; Altman et al. 2023)에 의존한다.

이 그래프 마이닝 알고리즘을 사용하여 그림 1에 소개된 의심 패턴을 열거한다. 서로 다른 크기의 패턴을 별도로 세어 세밀한 수준에서 열거한다. 엣지 분류 과제(AML)에서는 엣지가 포함된 패턴을 세고 이 개수를 추가 엣지 특성으로 추가한다. 노드 분류 과제(ETH)에서는 노드에 대해 같은 작업을 수행한다.

전체 특성 집합(원래 원시 특성과 추가 그래프 기반 특성)으로 개별 엣지(또는 노드)에 대해 LightGBM(Ke et al. 2017) 모델을 학습한다. 패턴 열거는 특정 시간 범위로 제한될 수 있다. 최상의 성능을 얻기 위해 각 데이터셋에 대해 범위를 (하이퍼파라미터로서) 최적화한다.

F.3 합성 패턴 탐지 실험

훨씬 더 큰 AML 및 ETH 데이터셋과 달리, 이웃을 부분 샘플링할 필요가 없다. 대신 모든 패턴을 찾을 수 있도록 모델에서 전체 3-홉 이웃을 사용한다. 더욱이, 6-사이클도 탐지할 수 있도록 6개의 GNN 층을 사용하였다.

F.4 AML 실험

엣지 분류 데이터셋이므로, 모든 GNN은 엣지 임베딩과 해당 종점 노드 임베딩을 입력으로 받는 최종 엣지 리드아웃(readout) 층을 사용한다. 이웃 샘플링을 사용하여 1-홉 및 2-홉 이웃을 각각 100개씩 샘플링한다. 사기성 노드 ID에 과적합하는 것을 최대한 피하기 위해 엣지와 노드 특성에서 노드 ID를 제거한다. 이 데이터셋을 생성하는 데 사용된 금융 범죄 패턴을 알고 있으므로, 그래프 마이닝 라이브러리를 사용하여 각 거래에 대해 정확히 이러한 의심 패턴을 열거한다. 다양한 크기의 degree-in/-out, fan-in/-out, 방향성 사이클, 무방향 사이클, 스캐터-개더 패턴이다. 바이클릭은 현재 라이브러리에 포함되어 있지 않으므로 제외한다. 낮은 IR 데이터셋의 경우, 해당 높은 IR 데이터셋의 하이퍼파라미터를 가져와 소수 클래스 가중치만 최적화하였다. 더 큰 클래스 불균형을 고려하기 위해 이것을 최적화해야 함을 발견하였다.

F.5 AML 런타임

표 5는 AML Small HI 데이터셋에서 실행할 때 GIN에 적응 기법을 추가하는 런타임 비용을 보여준다. GNN 모델의 크기는 동일하게 유지하였다. 2개의 GNN 층과 은닉 크기 64이다. 모든 모델은 Nvidia GeForce RTX 3090 GPU에서 실행되었다.

Model	TTT (s)	TPS (inference)
GIN	1876	53130
Individual Adaptations		
GIN+Ports	2007	58663
GIN+EgoIDs	1873	58533
GIN+ReverseMP	12510	21067
Cumulative Adaptations		
GIN+ReverseMP+Ports	13051	18763
+EgoIDs	12958	18700

표 5: AML Small HI 데이터셋에서 모든 GIN 기반 모델의 총 학습 시간(TTT)과 초당 거래 수(TPS). TPS 값은 평가 모드에서 측정되었다.

F.6 ETH 실험

노드 분류 데이터셋이므로, 모든 GNN은 해당 노드 임베딩만 입력으로 받는 최종 노드 리드아웃 층을 사용한다. 이웃 샘플링을 사용하여 1-홉 및 2-홉 이웃을 각각 1000개씩 샘플링한다.

LightGBM 베이스라인의 경우, 그래프 마이닝 라이브러리를 사용하여 AML과 동일한 그래프 기반 패턴을 열거하되 이번에는 노드에 대해 수행한다. 추가로, 베이스라인을 더욱 향상시키기 위해 거래 금액과 거래 간 시간의 기본 노드 통계(평균, 최댓값, 분산 등)를 계산한다. 통계 특성은 소수 클래스 F1 점수를 원시 및 그래프 기반 특성만 사용할 때의 27.1%에서 추가 통계 특성을 사용하면 50% 이상으로 크게 향상시킨다.

부록 G. 추가 실제 벤치마크

이론적 결과와 부분그래프 탐지 과제는 아키텍처 적응 기법의 범용적 잠재력을 보여준다. 실제로 우리가 고려하는 많은 부분그래프 패턴이 다른 영역과도 관련될 수 있다. 그러나 이러한 주장을 뒷받침하려면 실제 벤치마크 데이터셋에서 모델을 테스트하는 것이 필수적이다. 따라서 세 가지 실제 방향성 그래프 데이터셋을 가져와 이 3개 벤치마크의 최첨단(SOTA) 모델(Rusch et al. 2022)과 우리의 접근법을 비교하였다. 데이터셋 통계 요약은 표 6에서 볼 수 있다.

Dataset	# nodes	# edges	# node feats	# classes
Chameleon	2,277	36K	2,325	5
Squirrel	5,201	217K	2,089	5
Arxiv-Year	169K	1.17M	128	40

표 6: 실제 데이터셋 통계.

다양한 적응 기법을 적용한 GIN의 결과를 보고한다. 모델은 이전 실험과 같이 설정된다. Adam 옵티마이저를 사용하고 검증 정확도에 대한 조기 종료(인내 400)를 사용하여 모델을 2000 에폭 학습한다. 사전 정의된 범위 내에서 하이퍼파라미터를 최적화하기 위해 무작위 샘플링을 한 번 실행한다. 20개의 서로 다른 하이퍼파라미터 집합을 샘플링하고 최고 성능 모델을 선택한다. 데이터셋의 표준 데이터 분할을 사용한다. Chameleon과 Squirrel에는 고정 GEOM-GCN 분할(Pei et al. 2020)을 사용하고, Arxiv-Year에는 Lim et al. (2021)이 제공하는 분할을 사용한다. 10개의 서로 다른 데이터 분할에 대해 계산된 테스트 정확도의 평균과 표준편차를 보고한다. 결과는 표 7에서 볼 수 있다.

Model	Squirrel	Chameleon	Arxiv-Year
Grad. Gating (SOTA)	64.26 ± 2.38	71.40 ± 2.38	63.30 ± 1.84
GIN (GCN*)	35.94 ± 2.01*	51.29 ± 0.63	50.15 ± 0.26
+ReverseMP	59.84 ± 2.08*	66.36 ± 1.23	63.31 ± 0.45
+Ports	59.33 ± 2.40*	73.46 ± 0.97	68.12 ± 0.51
+EgoIDs	58.41 ± 2.49*	73.73 ± 1.03	68.04 ± 0.60

표 7: 세 가지 실제 방향성 그래프 데이터셋에서 최첨단(Rusch et al. 2022) 결과와의 비교.

3개 벤치마크 중 2개에서 SOTA를 능가한다. 특히 한 데이터셋에서는 항상 폭이 거의 6% 포인트이다.

부록 H. 추가 부분그래프 탐지 결과

H.1 추가 GNN 베이스라인

부분그래프 탐지를 염두에 두고 단순(루프나 평행 엣지가 없는) 무방향 그래프를 위해 개발된 "표현력 있는" GNN 아키텍처가 많이 있다. 예를 들어 PPGN(Maron et al. 2019)과 DropGIN(Papp et al. 2021) 등이다. 그러나 섹션 6의 기반 GNN 및 베이스라인에서 논한 바와 같이, 이들 중 어느 것도 방향성 멀티그래프를 위해 개발되지 않았으며, 안타깝게도 이들의 부분그래프 탐지 능력은 이 설정으로 이전되지 않는다. 예를 들어, 이 모델들은 엣지의 방향을 고려하지 않으므로 들어오는 이웃과 나가는 이웃을 구별할 수 없다. 이러한 아키텍처 중 일부는 이 설정에 맞게 적응될 수 있으며, 어떤 경우에는 우리가 제안한 적응 기법이 그렇게 할 수 있는 가능한 접근법을 제공한다. 우리의 주장을 뒷받침하기 위해 부분그래프 탐지 실험에 더 많은 베이스라인을 추가하였으며,

결과는 표 8을 참조하라. 가능한 경우 제안된 적응 기법에 기반한 적응 버전도 포함한다. 이 아키텍처들은 적응 없이는 실제로 좋은 성능을 내지 못한다.

Model	C4	C5	C6	S-G	B-C
DropGIN	16.92	26.02	45.17	48.87	51.59
R-GCN	65.57	64.35	71.27	59.23	55.61
PPGN	57.93	48.74	41.15	54.23	54.98
Multi-DropGIN	99.09	97.08	94.93	96.98	94.27
Multi-R-GCN	85.20	74.08	74.30	82.37	67.16
Multi-PPGN*	98.11	92.49	86.69	68.74	62.78
Multi-PNA	99.49	97.46	88.75	99.07	96.77

표 8: 추가 부분그래프 탐지 결과(F1 점수). 비교를 위해 Multi-PNA가 포함되어 있다. * 포트 번호 없음.

H.2 복잡한 패턴과 데이터셋 크기

부분그래프 탐지 과제에서 데이터셋 크기의 영향을 측정하기 위해 추가 실험을 수행하였다. 이 실험은 모든 적응 기법을 적용한 GIN 모델로 제한한다. 또한 거의 완벽하게 해결되지 않은 더 "복잡한" 과제, 즉 C4, C5, C6, 스캐터-개더, 바이클릭 탐지로 제한한다. 1024에서 16384까지 2배씩 증가하는 데이터셋 크기를 사용한다. 다른 모든 그래프 생성기 설정은 일정하게 유지한다. 특히 동일한 평균 차수와 평균 반경을 사용한다.

결과는 그림 9에서 볼 수 있고, 구체적인 값은 표 9에서 찾을 수 있다. 학습 데이터셋 크기를 늘리면 모든 복잡한 과제에서 점수가 점진적으로 향상되며, 6-사이클 탐지조차 소수 클래스 F1 점수 97% 이상에 도달함을 볼 수 있다. 별표(*)가 붙은 결과는 더 작은 과제 집합으로 제한하는 것이 성능을 크게 향상시킴을 보여준다.

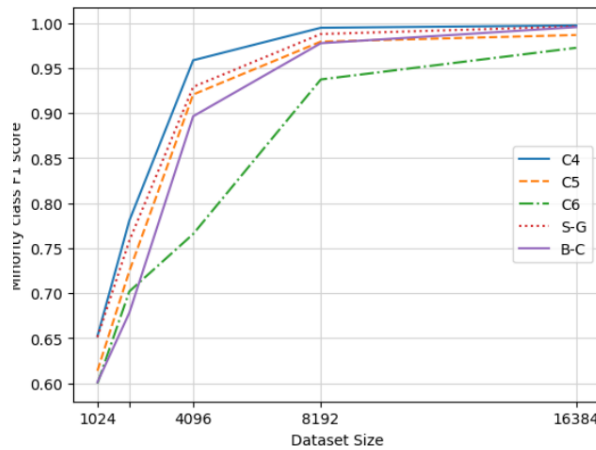


그림 9: 학습 데이터셋 크기가 증가함에 따른 "복잡한" 하위 과제에서의 Multi-GIN 성능.

No. Training Nodes	C4	C5	C6	S-G	B-C
1024	65.22 ± 3.34	61.38 ± 7.25	59.97 ± 7.30	65.12 ± 2.83	60.11 ± 2.86
2048	78.06 ± 6.19	72.44 ± 4.48	70.19 ± 2.49	75.82 ± 4.77	67.82 ± 1.53
4096	95.85 ± 0.86	92.06 ± 1.81	76.57 ± 0.96	92.90 ± 1.29	89.64 ± 1.74
8192*	97.46 ± 0.43	91.60 ± 1.50	84.23 ± 1.12	97.42 ± 0.85	94.33 ± 0.95
8192	99.46 ± 0.05	97.92 ± 0.26	93.72 ± 0.75	98.77 ± 0.18	97.75 ± 0.21
16384	99.72 ± 0.06	98.66 ± 0.15	97.24 ± 0.19	99.61 ± 0.08	99.53 ± 0.07

표 9: 다양한 데이터셋 크기에서 "복잡한" 부분그래프 탐지 과제의 소수 클래스 F1 점수. 모든 실험에 Multi-GIN이 사용된다. *는 이 결과들이 모든 부분그래프 과제를 한 번에 학습한 표 1에서 가져왔음을 나타낸다. 다른 모든 실험은 표시된 하위 과제로 제한되었다.

사이클의 경우, 이 실험을 길이 10의 사이클과 65536개 노드의 학습 데이터셋으로 더 확장하였다. 이 결과는 표 10에서 볼 수 있으며, 가장 큰 학습 데이터셋에서 모든 사이클 크기가 소수 클래스 F1 점수 98% 이상에 도달하는 유사한 경향을 보여준다.

No. Training Nodes	C2	C3	C4	C5	C6	C7	C8	C9	C10
2048	97.53	89.72	84.71	80.60	75.48	81.34	91.51	93.57	93.39
4096	99.51	96.49	95.23	89.75	85.71	85.85	90.69	94.52	96.09
8192	99.83	99.78	99.13	97.17	93.58	89.93	91.87	94.92	96.64
16384	99.96	99.95	99.75	99.01	97.46	95.89	95.15	96.45	97.64
32768	99.99	100.00	99.93	99.62	98.76	97.98	97.93	98.10	98.30
65536	100.00	100.00	99.96	99.80	99.36	98.89	98.73	98.86	98.83

표 10: 다양한 학습 데이터셋 크기에서 증가하는 길이의 사이클 탐지에 대한 소수 클래스 F1 점수. 모든 실험에 Multi-GIN이 사용된다.

H.3 고유 노드 ID를 사용한 부분그래프 탐지

아래에 추가 부분그래프 탐지 결과를 포함한다. 먼저, 모든 GIN 기반 모델에 랜덤 고유 노드 ID를 입력의 일부로 추가하여 부분그래프 탐지 실험을 다시 실행한다. 결과는 표 12에서 볼 수 있다. 이 실험은 이론적 표현성이 항상 더 나은 실제 결과로 이어지지 않는 것을 보여준다. 이 표의 첫 행을 표 1의 첫 행과 비교하면, 랜덤 노드 ID가 범용적이긴 하지만 부분그래프 탐지에는 효과적이지 않음이 드러난다. 표의 나머지 부분은 적응 기법 추가의 효과를 보여준다. 경향은 같고 결과는 표 1과 유사하지만, 노드 ID를 사용하면 대부분의 점수가 실제로 약간 낮다. 특히 복잡한 패턴에서 높은 점수를 얻으려면 (역방향 메시지 전달에 더해) 포트 번호와 ego ID가 여전히 필요하다는 점에 주목하라. 이는 노드 ID가 이 두 특성 중 하나를 쉽게 제공할 수 있음에도 불구하고 신경망이 이 정보를 효과적으로 추출하고 사용할 수 없음을 강조한다. 결과는 적응 기법이 이 과제에 적합한 귀납 편향(inductive bias)을 제공함을 나타낸다.

Model	deg-in	deg-out	fan-in	fan-out	C2	C3	C4	C5	C6	S-G	B-C
GIN	99.91	44.68	95.26	40.72	25.05	55.30	46.73	43.61	47.32	65.97	63.58
GIN+EgoIDs	99.82	52.56	95.67	49.78	98.67	95.78	54.56	45.95	48.01	69.00	64.93
GIN+Ports	98.88	44.49	99.16	40.71	27.84	55.73	48.03	43.30	47.78	66.46	63.04
GIN+ReverseMP	98.58	99.24	93.19	94.23	38.18	62.62	69.74	69.08	74.07	65.83	63.67
+Ports	98.36	98.31	99.01	99.37	37.13	61.88	72.12	71.04	76.47	65.00	66.89
+EgoIDs (Multi-GIN)	99.31	99.05	99.36	99.37	98.28	97.97	95.92	87.82	78.15	96.83	91.26

표 12: 랜덤 고유 정수 노드 ID를 사용한 부분그래프 탐지 결과.

H.4 압축된 멀티그래프를 사용한 부분그래프 탐지

둘째로, 방향성 멀티그래프 입력을 엣지 가중치가 원래 멀티그래프의 평행 엣지 수를 나타내는 가중 엣지를 가진 방향성 그래프로 압축할 수 있는지 탐구한다. 모든 GIN 기반 모델로 부분그래프

탐지 실험을 다시 실행한다. 압축된 멀티그래프 실험의 결과는 표 11에서 찾을 수 있다. 결과는 원래 멀티그래프를 사용하는 것과 매우 유사하다. 그러나 중요하게도, 이 과제들을 해결하는 데 포트 번호가 더 이상 필요하지 않다는 점에 주목하라. 예를 들어, 이제 포트 없이도 fan-in을 완벽하게 해결할 수 있으며, 포트 번호는 GIN이든 역방향 메시지 전달을 갖춘 GIN이든 추가해도 점수를 유의미하게 높이지 않는다. 이는 놀라운 일이 아닌데, 이 과제들은 (필요에 따라 역방향 메시지 전달과 ego ID가 주어지면) 가중 방향성 그래프에서 쉽게 해결될 수 있기 때문이다. 실제로 이런 방식으로 멀티그래프의 평행 엣지를 압축하는 것이 일부 멀티그래프 과제에는 유효한 선택지일 수 있다. 그러나 엣지가 과제에 중요한 비가산적(non-additive) 특성을 갖는 순간, 이것은 더 이상 선택지가 아닐 수 있다. 귀중한 정보를 잃지 않고 평행 엣지의 특성을 요약하는 것이 불가능할 수 있다.

Model	deg-in	deg-out	fan-in	fan-out	C2	C3	C4	C5	C6	S-G	B-C
GIN	99.63	45.70	99.49	42.15	35.48	57.04	51.07	45.66	48.32	69.38	63.85
GIN+EgoIDs	99.90	51.51	99.64	49.92	98.89	93.71	57.89	49.08	49.78	67.27	62.88
GIN+Ports	99.84	42.33	99.51	38.61	33.72	55.77	55.43	47.25	46.79	68.89	65.68
GIN+ReverseMP	98.64	98.73	97.74	98.83	40.09	61.10	72.34	70.27	74.55	66.44	66.26
+Ports	99.03	99.44	99.35	99.37	43.15	63.15	68.20	68.29	73.67	68.02	66.87
+EgoIDs (Multi-GIN)	99.68	99.71	99.76	99.52	99.02	99.13	97.93	93.88	82.83	99.47	98.81

표 11: 평행 엣지를 단일 가중 엣지로 결합한 부분그래프 탐지 결과.

부록 I. 상세 AML 결과

AML 데이터셋의 사기 거래 중 일부에는 정답 자금세탁 패턴 레이블이 있다. 나머지 (패턴 레이블이 없는) 불법 거래는 특정 자금세탁 패턴에 속하지 않는다. 따라서 이 거래들에 대해서는 더 낮은 재현율 점수를 예상할 수 있다. 이 패턴 레이블을 사용하여 각 패턴 유형별로 개별 재현율 점수를 계산하고, 이 결과를 AML Small HI는 표 13에, AML Small LI는 표 14에 제시한다.

Model	Fan-in	Fan-out	Cycle	S-G	G-S	B-C	none
GIN (Xu et al. 2018; Hu et al. 2019)	40.65	64.89	44.44	51.46	47.24	34.85	4.31
GIN+ReverseMP (Jaume et al. 2019)	58.54	80.92	40.40	72.80	68.50	36.36	2.43
+Ports	67.48	91.60	48.48	82.85	84.51	51.52	1.08
+EgoIDs (Multi-GIN)	87.80	80.15	50.51	89.96	86.35	51.52	1.89
Multi-GIN+EU	88.62	95.42	57.58	88.28	94.49	57.58	1.35
Multi-PNA	85.27	92.12	61.62	92.72	92.65	51.55	0.54

표 13: 정답 패턴 레이블을 사용할 수 있는 AML Small HI 데이터셋에서 자금세탁 패턴별 재현율 점수. S-G는 Scatter-Gather, G-S는 Gather-Scatter, B-C는 Bipartite를 나타내고, none은 표준 자금세탁 패턴에 속하지 않는 불법 거래를 나타낸다. 정답 자금세탁 패턴의 크기는 다양하다.

Model	Fan-in	Fan-out	Cycle	S-G	G-S	B-C	none
GIN (Xu et al. 2018; Hu et al. 2019)	0.00	60.87	11.90	20.34	41.43	2.38	1.45
GIN+ReverseMP (Jaume et al. 2019)	14.29	36.96	26.19	37.29	34.29	28.57	5.58
+Ports	35.71	76.09	42.86	54.24	54.29	42.86	3.31
+EgoIDs (Multi-GIN)	42.86	56.52	28.57	45.76	51.43	40.48	0.83
Multi-GIN+EU	42.86	84.78	40.48	52.54	70.00	35.71	0.83
Multi-PNA	71.43	95.65	33.33	71.19	94.29	64.29	0.62

표 14: 정답 패턴 레이블을 사용할 수 있는 AML Small LI 데이터셋에서 자금세탁 패턴별 재현율 점수.

표에서 볼 수 있듯이, 우리 모델은 대부분의 세탁 패턴을 식별하여 매우 높은 개별 재현율 점수를 기록하지만, 사이클과 이분(bipartite) 재현율 점수는 더 개선될 수 있다. 이는 최고 성능 GNN 모델이 두 개의 메시지 전달 층만 사용하여 더 긴 사이클 탐지가 불가능했기 때문일 가능성이 높다. 층 수를 늘리면 전체 F1 점수를 높일 수 있지만 런타임 측면에서 상당한 비용이 따른다.

또한 특정 자금세탁 패턴에 속하지 않는 불법 거래(none)의 재현율 점수가 0%에 가깝다는 것을 즉시 알 수 있다. 이것이 아마도 세탁 거래의 71%가 세탁 패턴에 속하지 않는 Small LI의 낮은 총합 소수 클래스 F1 점수(표 2)를 설명할 것이다. Small HI의 해당 값은 38%에 불과하다. 이러한 통찰은 제안된 접근법이 자금세탁 패턴 탐지에 얼마나 효과적인지를 보여준다.